Workshop 1 - Web

java springboot Http Rest Api + Http Client(s)

TD Outline

TD Goals = code & debug yourself a basic Http client-server

1/ install "Java JDK"
2/ install IDE : "IntelliJ IDEA"
3/ setup your app, using spring "initializr" click+download
4/ run your app
5/ open http://localhost:8080
6/ inspect network requests
7/ test Http requests

TD Outline

TD Goals = code & debug yourself a basic http client-server

1/ install "Java JDK"
2/ install IDE : "IntelliJ IDEA"
3/ setup your app, using spring "initializr" click+download
4/ run your app
5/ open http://localhost:8080
6/ inspect network requests
7/ test Http requests

Pre-Requisite 1 : Install Java JDK

JDK is Open-Source, but pre-built binaries may be licenced if downloading from oracle.com

on linux => "apt-get install default-jdk"

on Mac => "brew install openjdk"

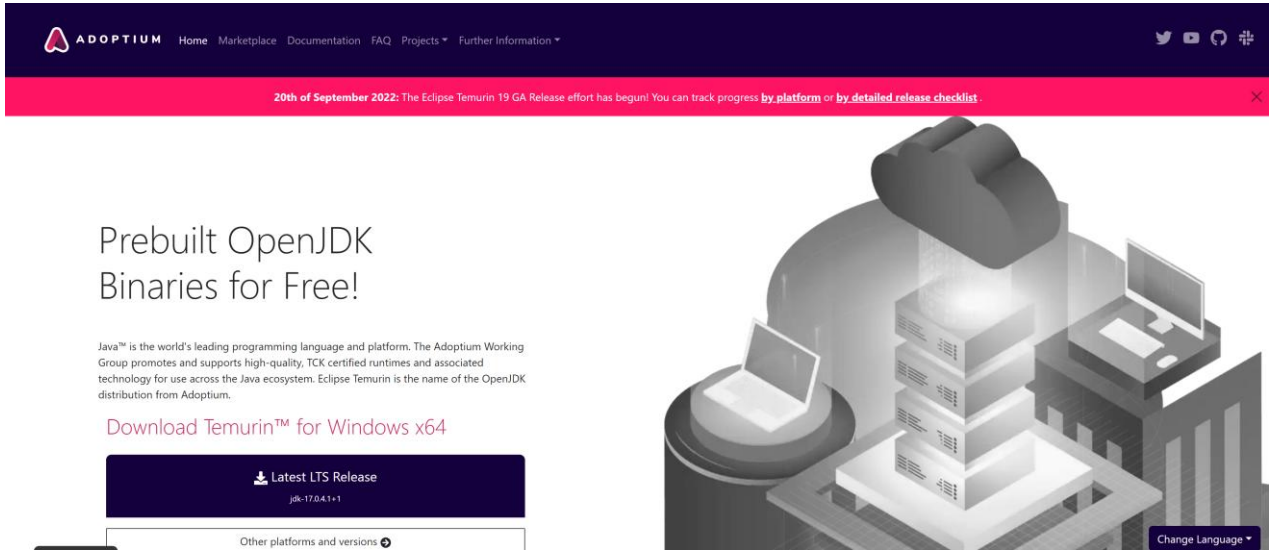
on Windows => download adoptium temurin .msi installer
+ execute (see next slides)

Step 1 : Install Java JDK, download (~170 Mo)

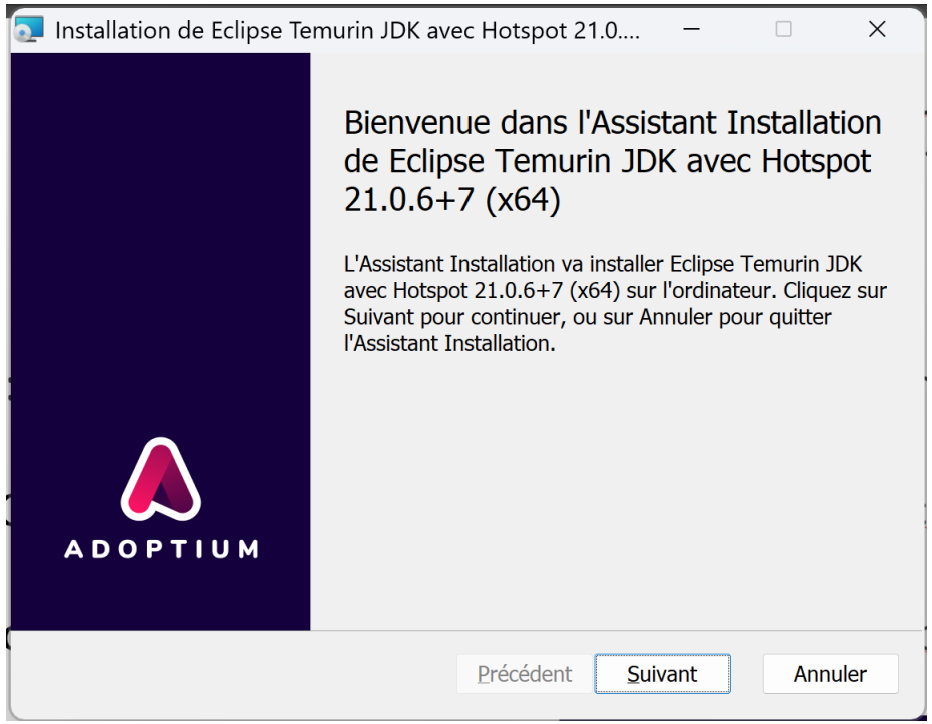
JDK is Open-Source, but pre-built binaries may be licenced if downloading from oracle.com

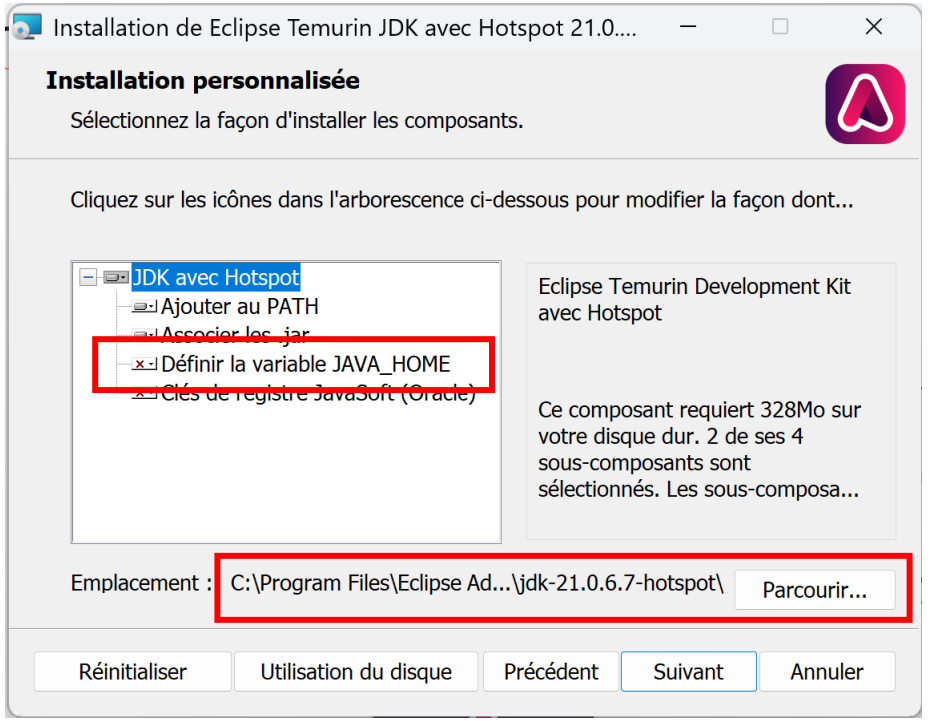
Download from « adoptium » (previously « adoptOpenJdk »)

https://adoptium.net/



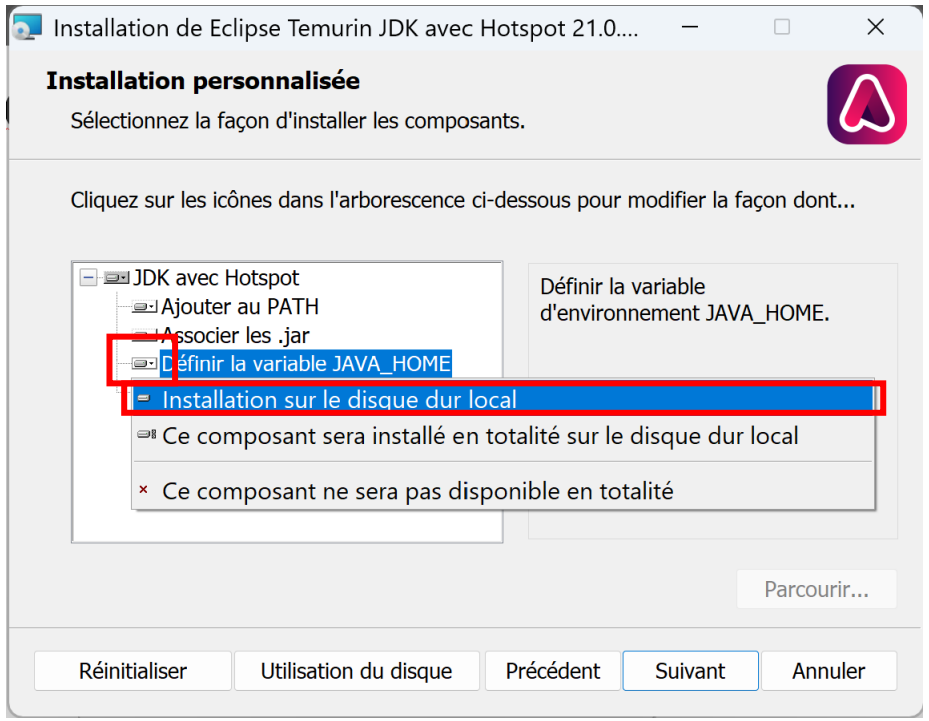
Installing ...

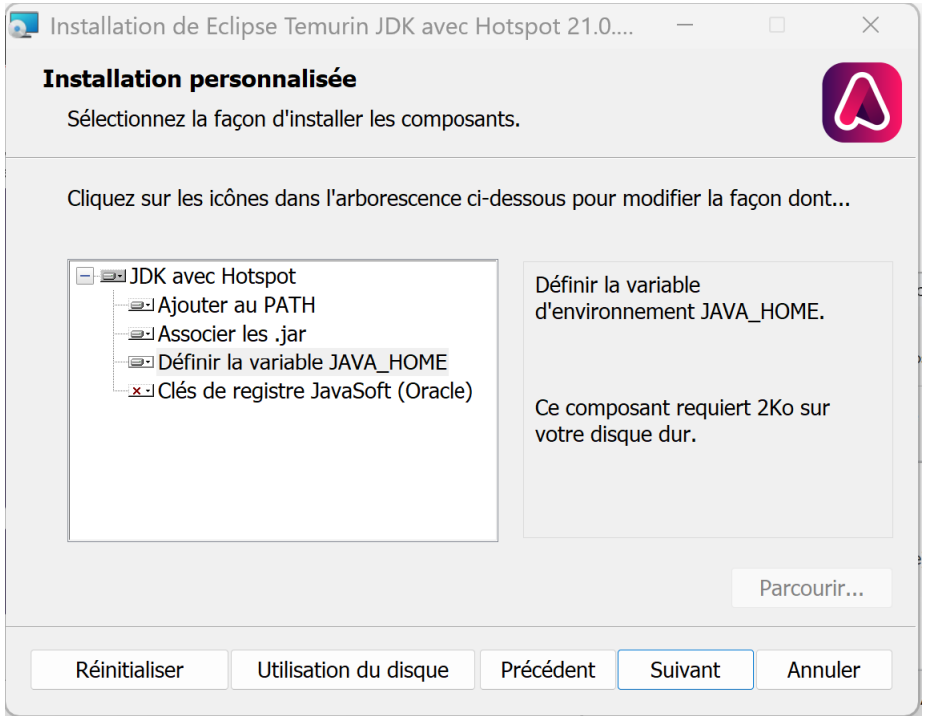




You may change defaults ... see next

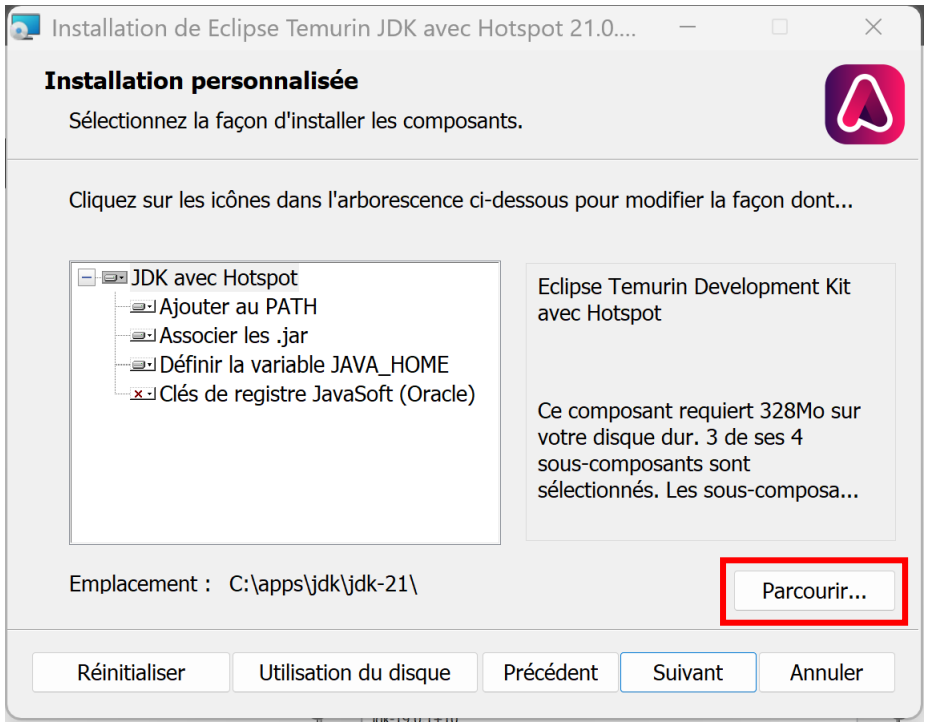
Optional :
defining env variable JAVA_HOME

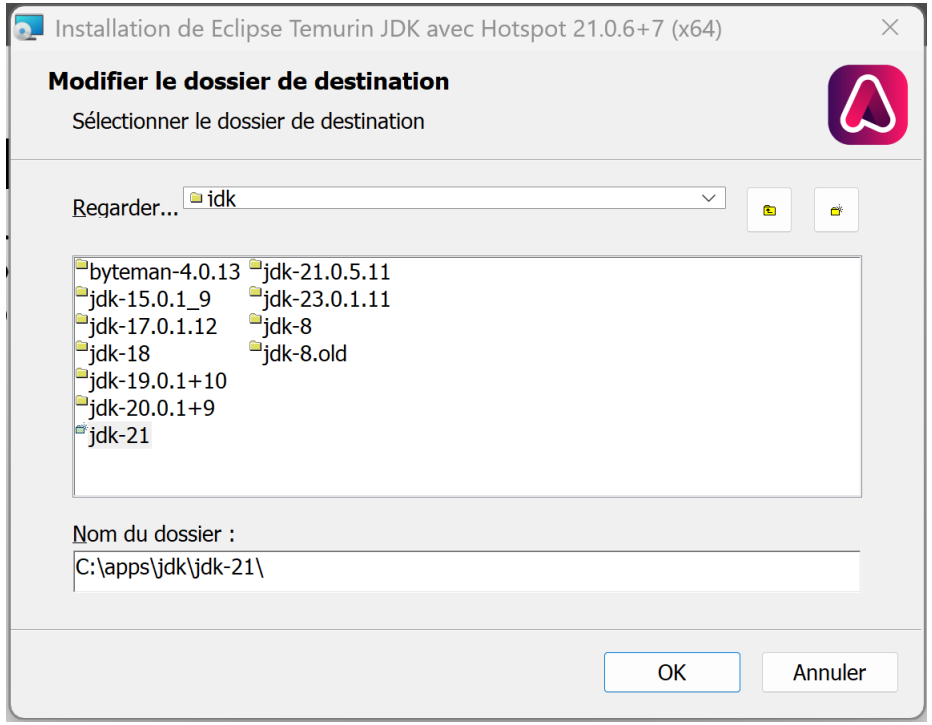




Optional

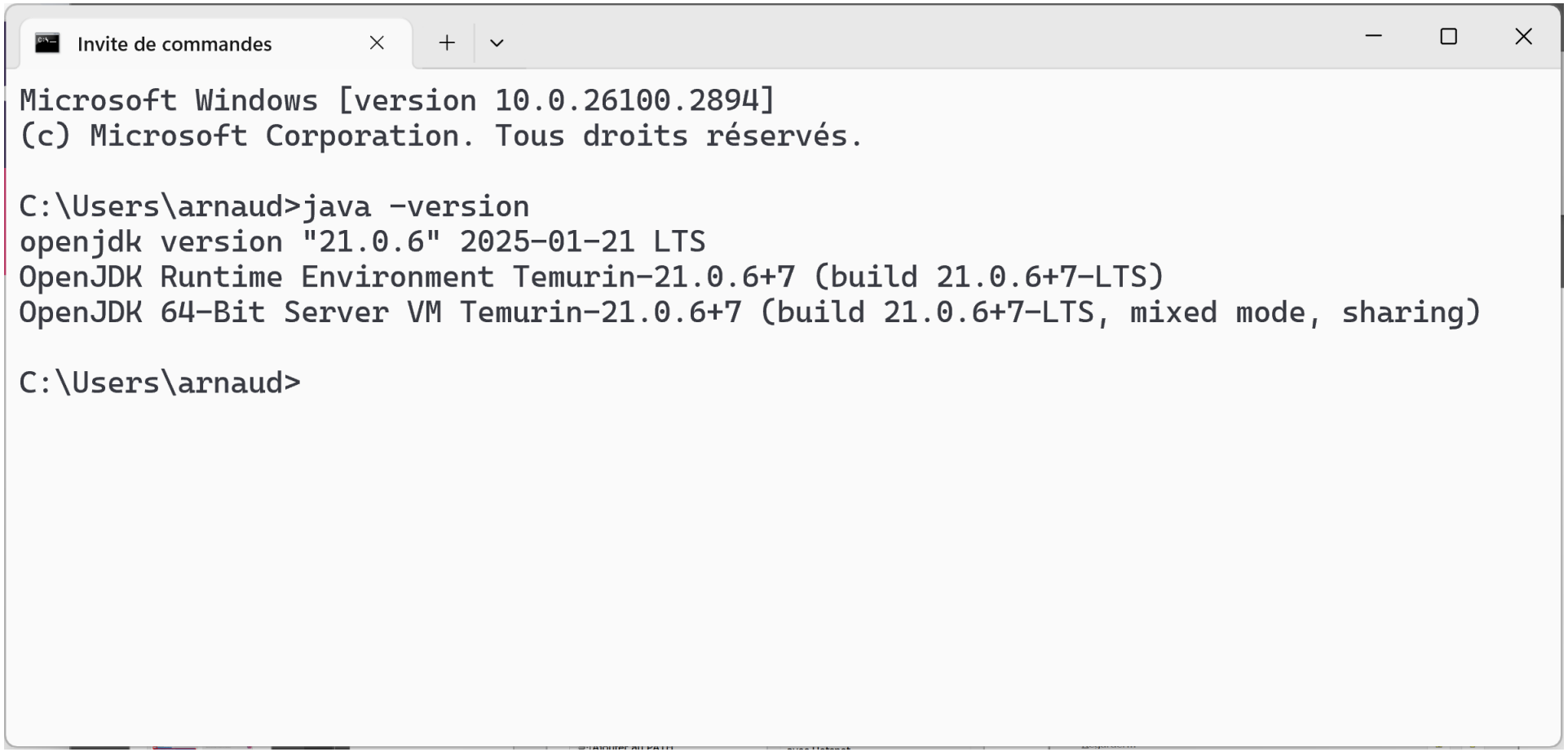
Changing directory "c:\apps\jdk"
(avoid long names, and space in dir name
so avoid "Program & Files\...")





Check Java installation OK

C:> java -version



TD Outline

TD Goals = debug yourself a basic Http client-server

1/ install "Java JDK"
2/ install IDE : "IntelliJ IDEA"
3/ setup your app, using spring "initializr" click+download
4/ run your app
5/ open http://localhost:8080
6/ inspect network requests
7/ test Http requests

Pre-Requisite 2 : Install IDE "IntelliJ IDEA"

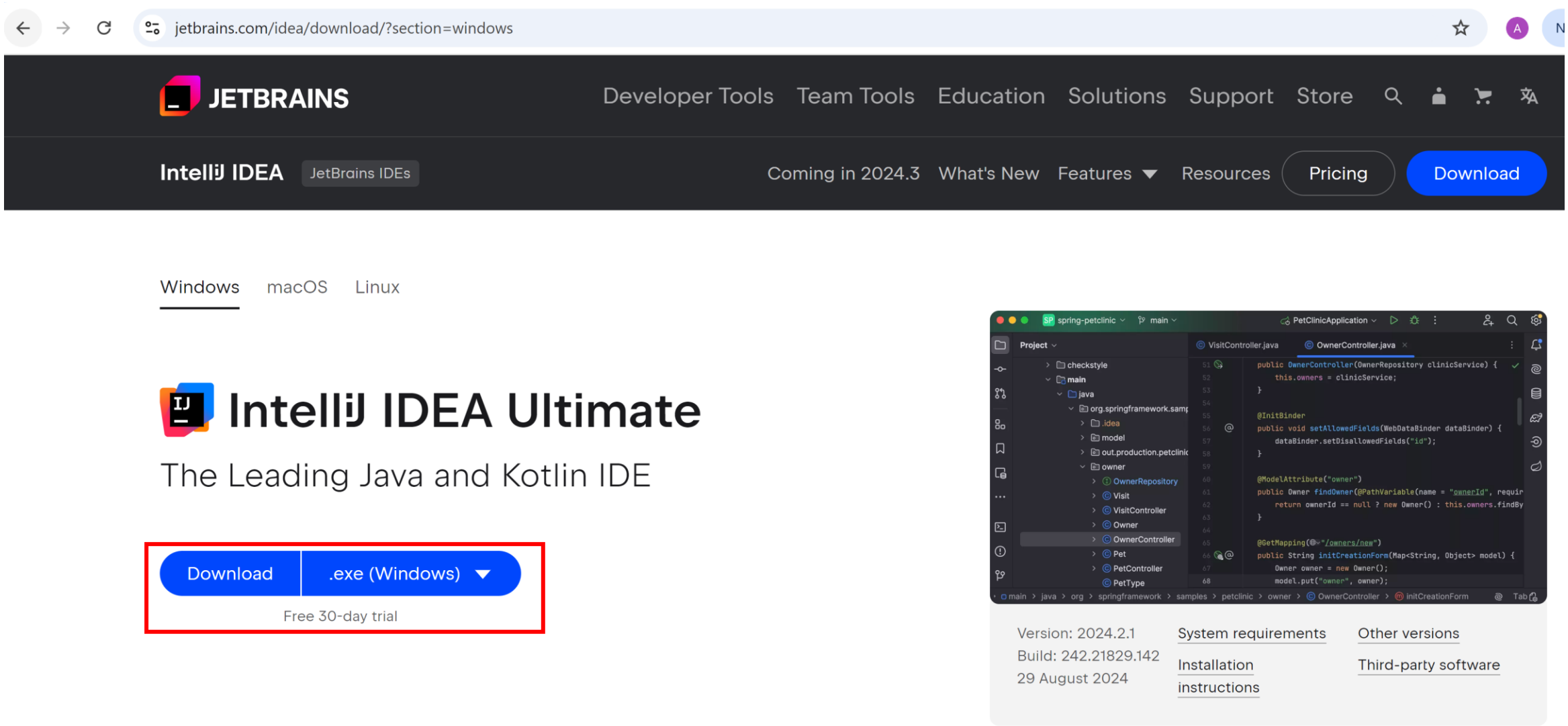
IntelliJ IDEA is the recommended IDE to use

it is the mostly used IDE in the world

100% ready and easy to use for java+maven+http+web+...

Download IntelliJ IDEA

https://www.jetbrains.com/idea/download



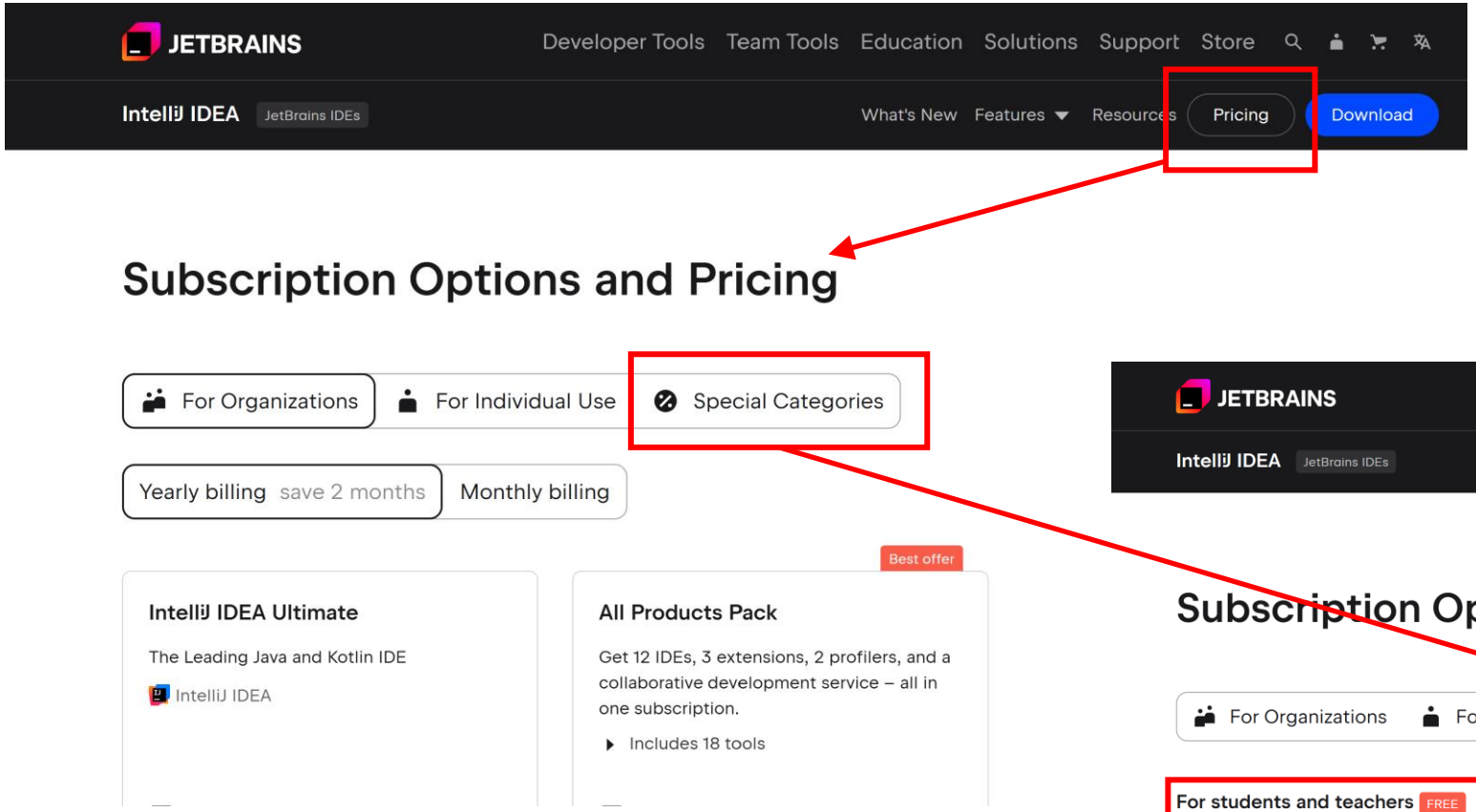
Free 30-days trial -OR- register free 1 year

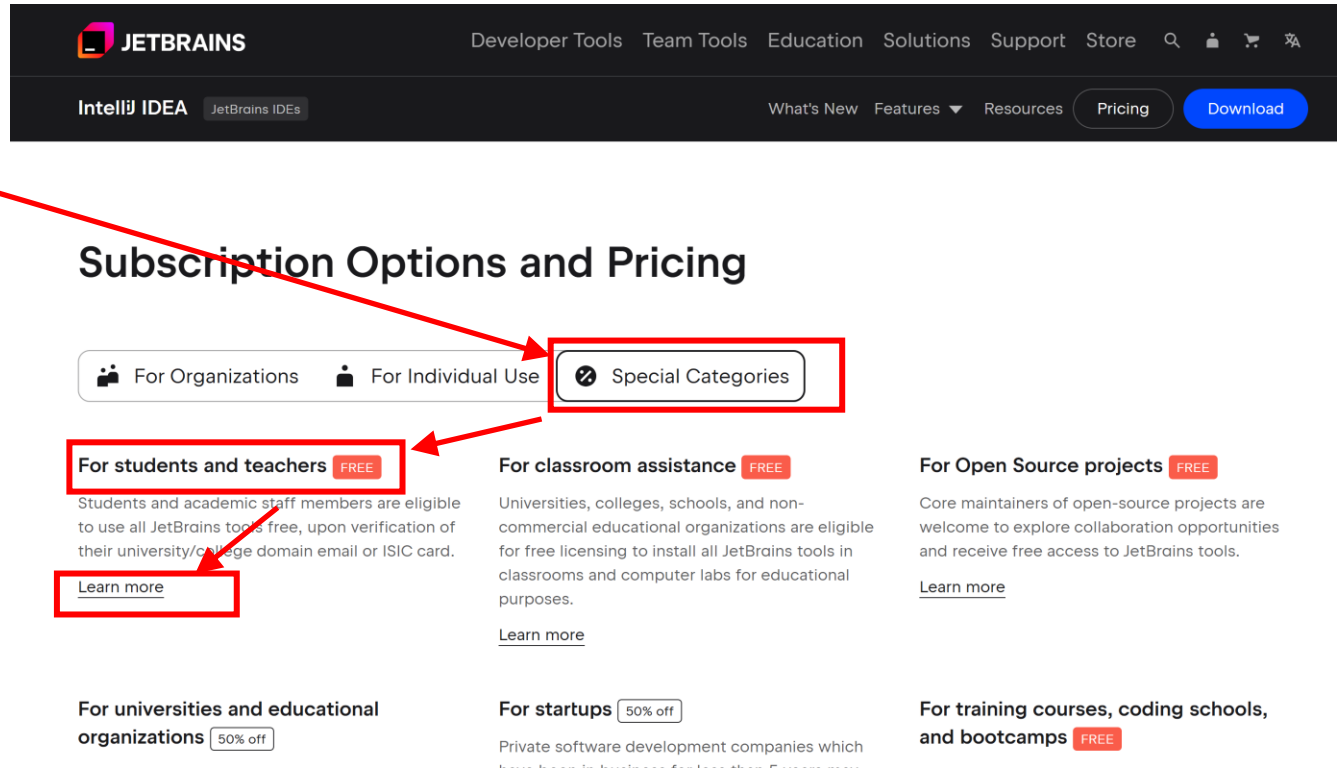
version "Community" is FREE (but supports only java)

version "Ultimate" is much more complete
... and is FREE for students
... is paid by most companies for all developpers who ask

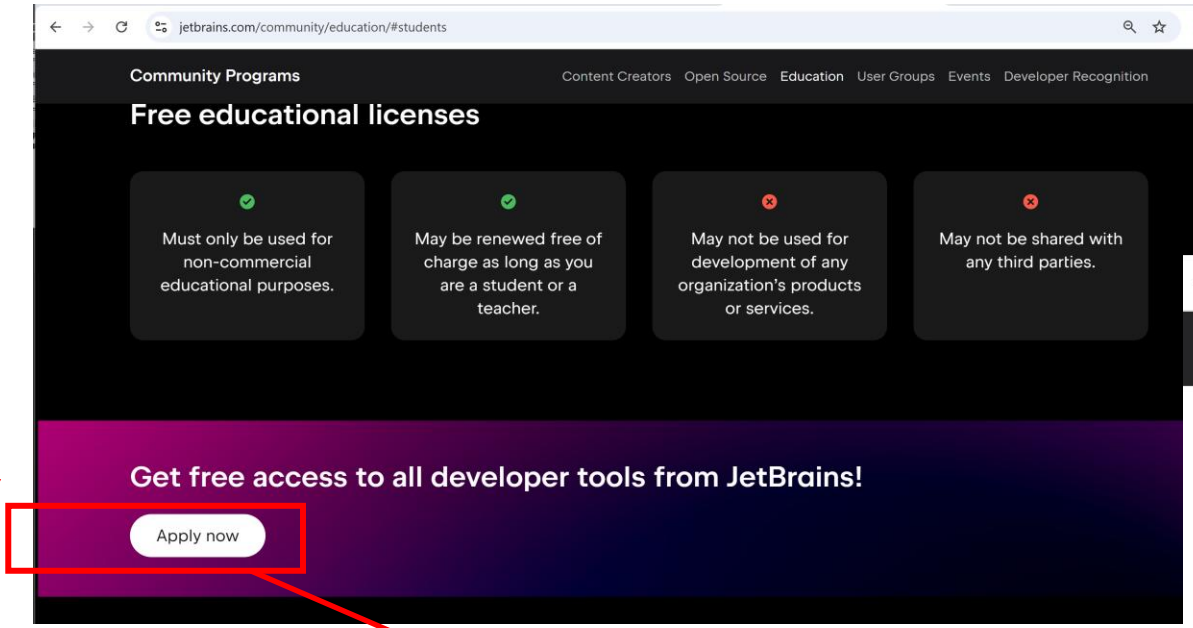
For doing today TD, you can just use "30-day trial"
if you have time (or later at home), you can fill the registration page with your "@esilv" email (see next slides)

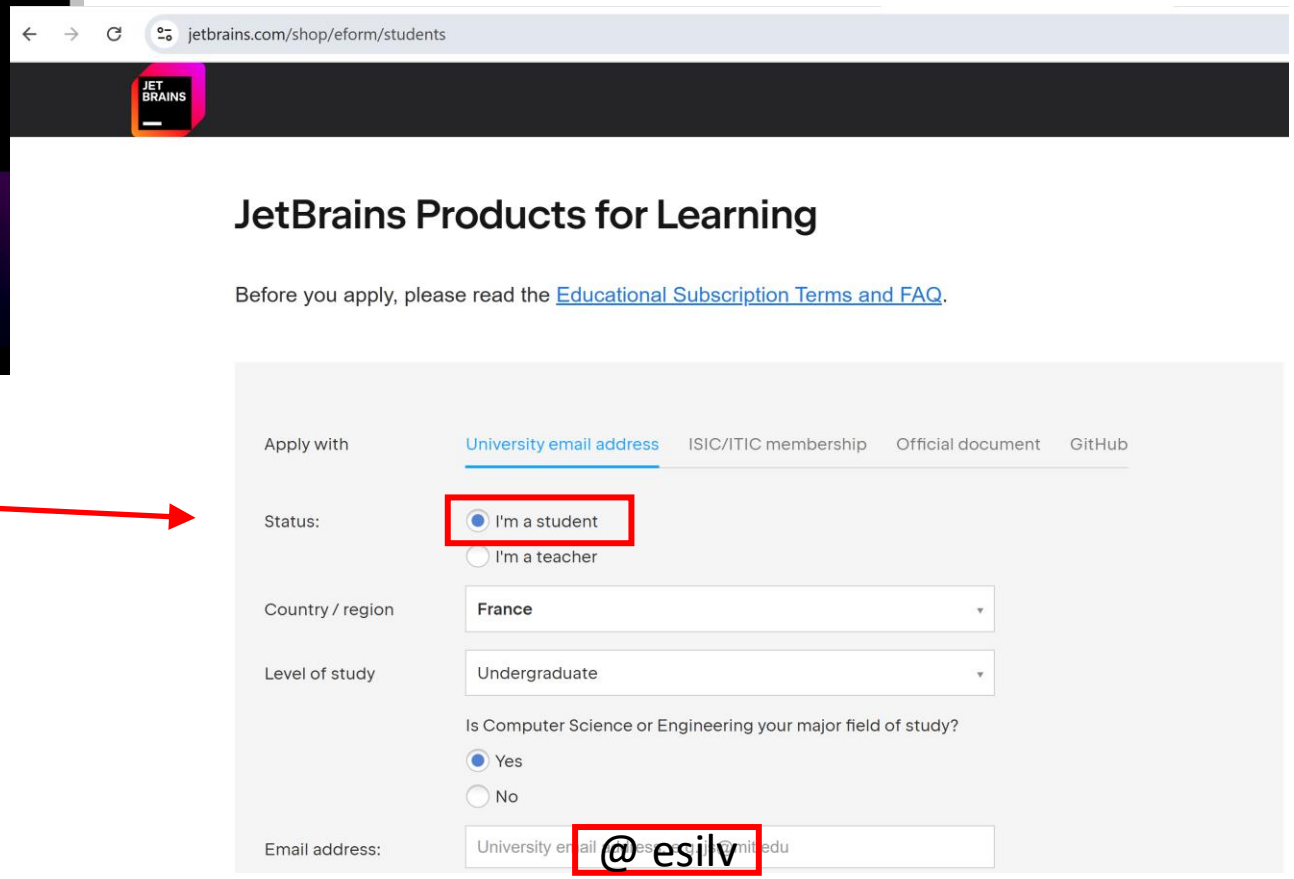
Register IntelliJ IDEA "Ultimate" as Student





Steps …


jetbrains.com/community/education/#students
Community Programs
Content Creators
Open Source
Education
User Groups
Events
Developer Recognition
Free educational licenses
Must only be used for non-commercial educational purposes.
May be renewed free of charge as long as you are a student or a teacher.
May not be used for development of any organization's products or services.
May not be shared with any third parties.
Get free access to all developer tools from JetBrains!
Apply now

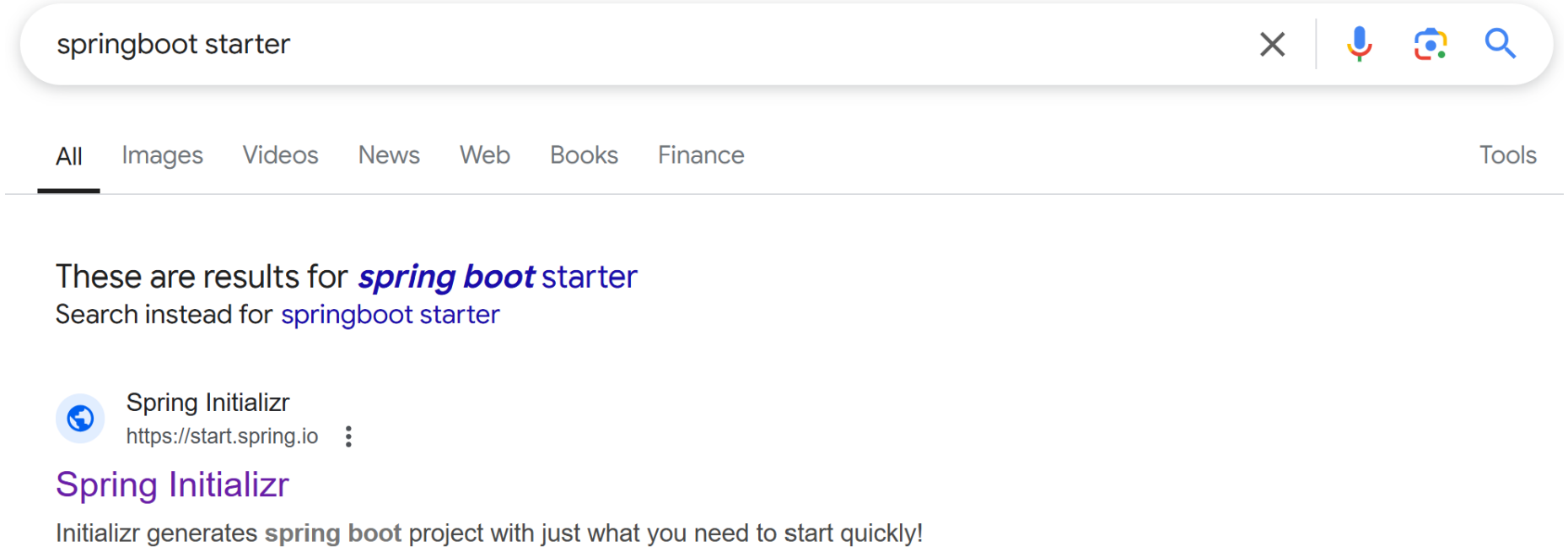

jetbrains.com/shop/eform/students
JET BRAINS
JetBrains Products for Learning
Before you apply, please read the Educational Subscription Terms and FAQ.
Apply with
University email address
ISIC/ITIC membership
Official document
GitHub
Status:
I'm a student
I'm a teacher
Country / region
France
Level of study
Undergraduate
Is Computer Science or Engineering your major field of study?
Yes
No
Email address:
@esilv

TD Outline

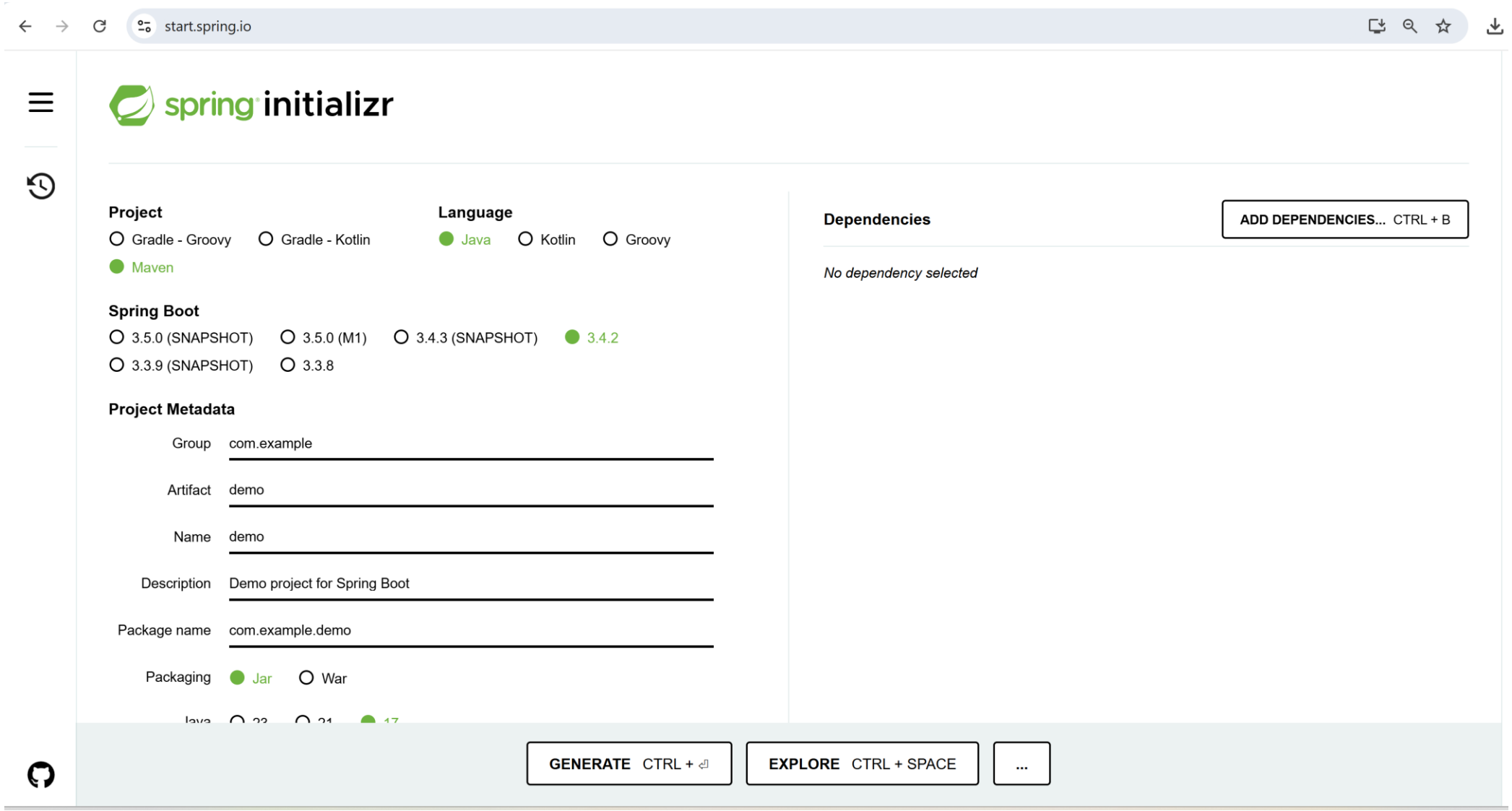
TD Goals = debug yourself a basic Http client-server

1/ install "Java JDK"
2/ install IDE : "IntelliJ IDEA"
3/ setup your app, using spring "initializr" click+download
4/ run your app
5/ open http://localhost:8080
6/ inspect network requests
7/ test Http requests

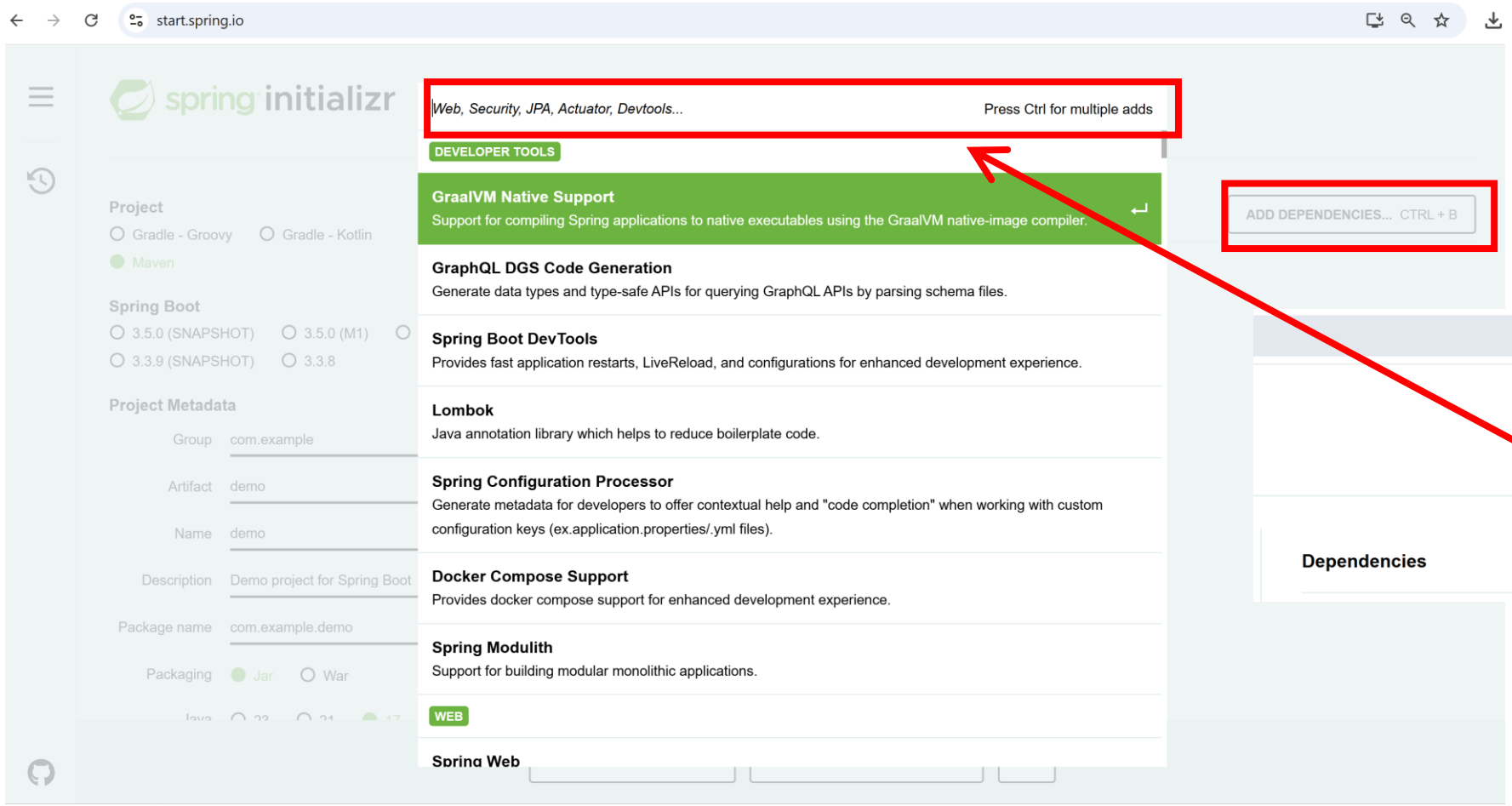
Using springboot starter aka "Spring Initializr" https://start.spring.io

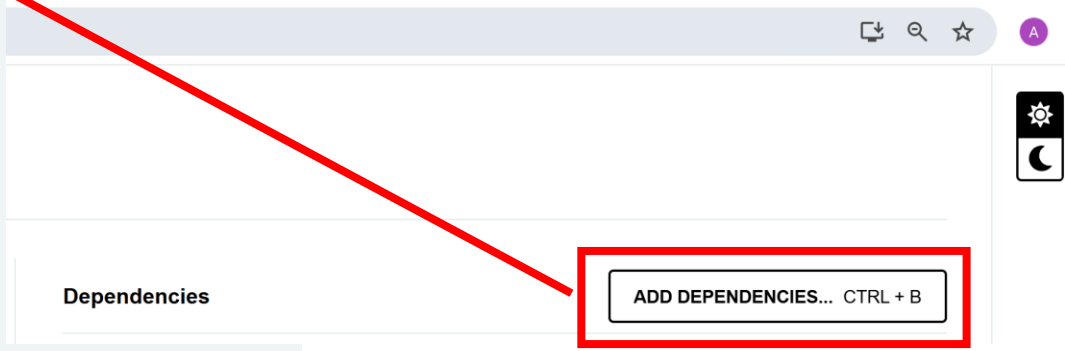


Open Springboot Initializr https://start.spring.io

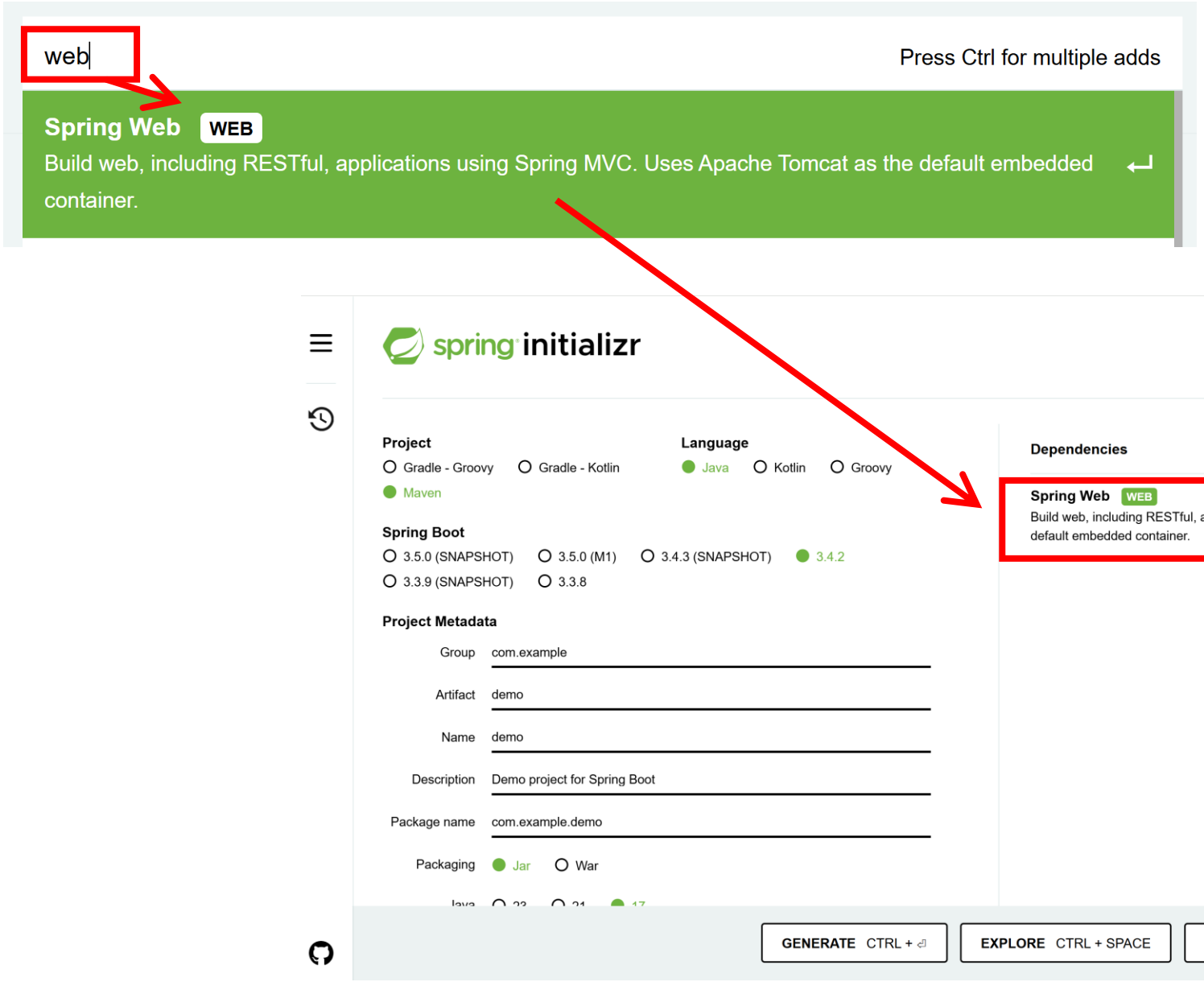


Click "ADD DEPENDENCIES..."

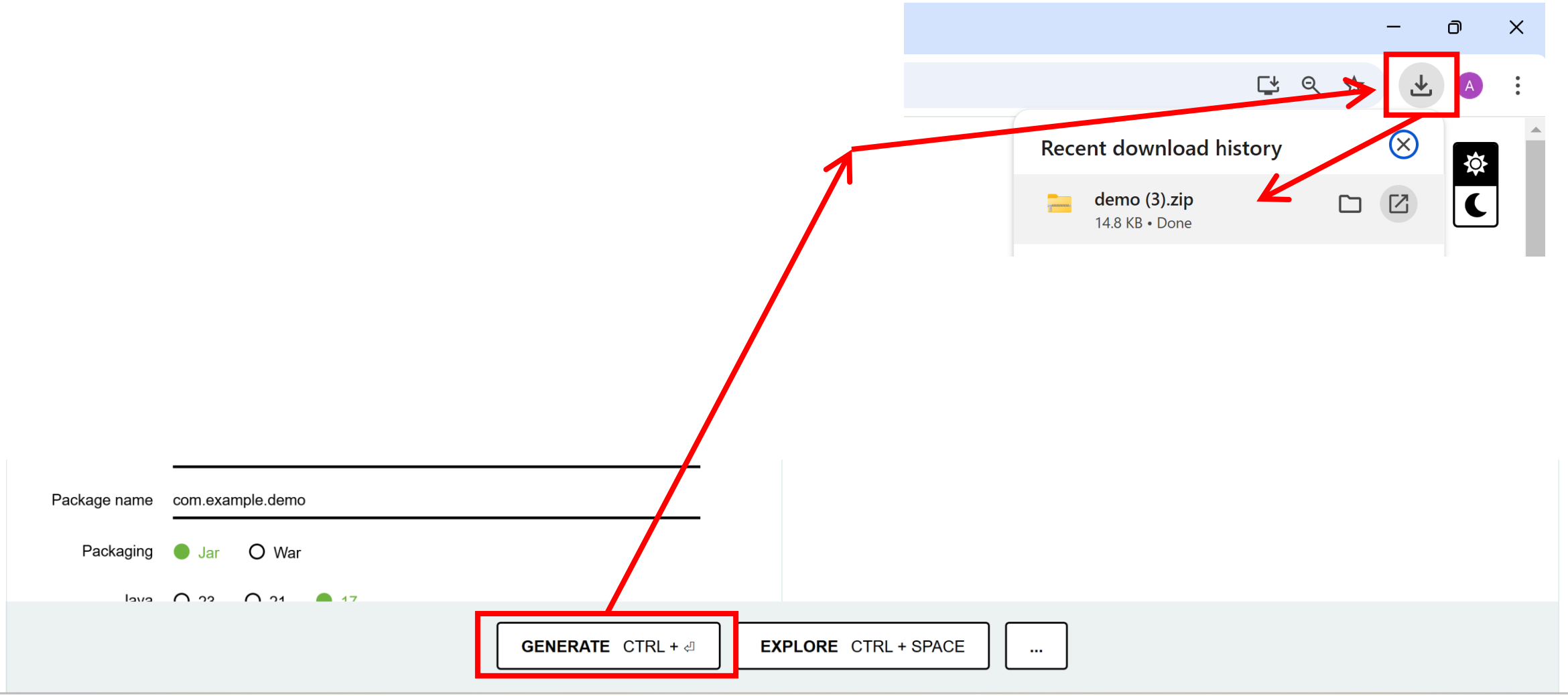




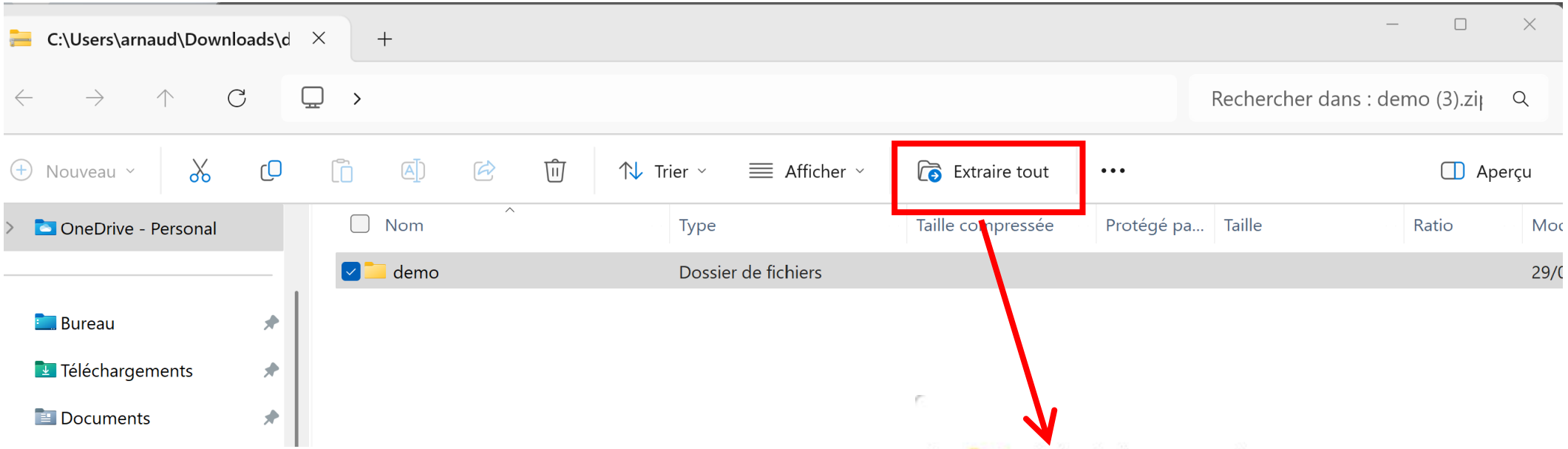
Add "web"

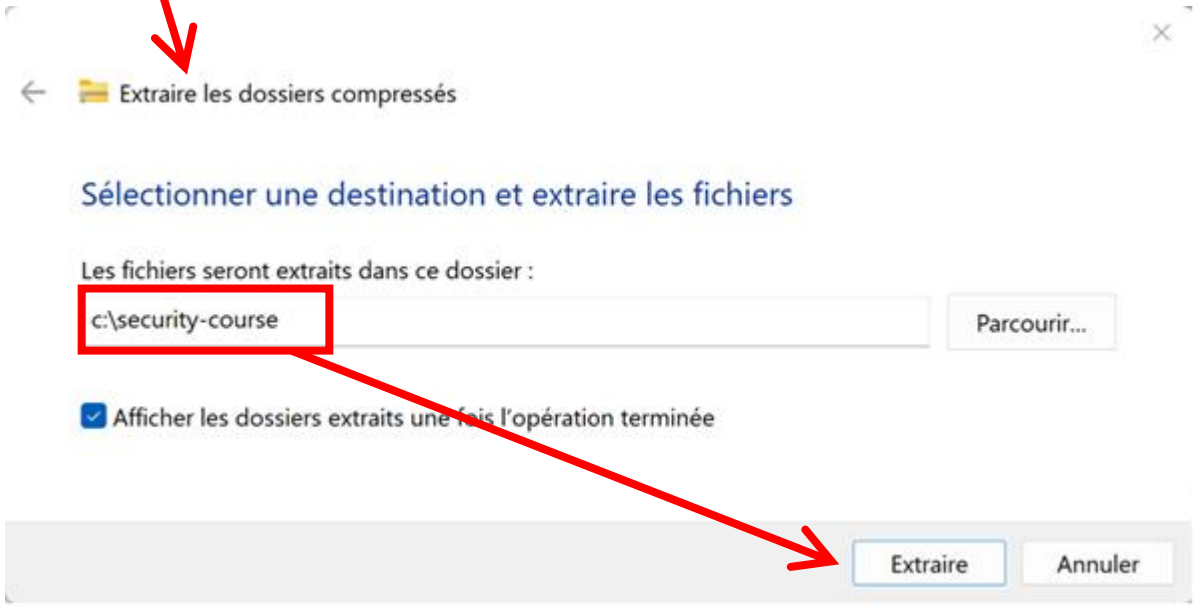

web
Press Ctrl for multiple adds
Spring Web
WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
spring initializr
Project
Gradle - Groovy
Gradle - Kotlin
Maven
Language
Java
Kotlin
Groovy
Spring Boot
3.5.0 (SNAPSHOT)
3.5.0 (M1)
3.4.3 (SNAPSHOT)
3.4.2
3.3.9 (SNAPSHOT)
3.3.8
Project Metadata
Group
com.example
Artifact
demo
Name
demo
Description
Demo project for Spring Boot
Package name
com.example.demo
Packaging
Jar
War
Dependencies
ADD DEPENDENCIES... CTRL + B
Spring Web
WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
GENERATE CTRL + ⏎
EXPLORE CTRL + SPACE
...

Then Click "GENERATE", open downloaded zip

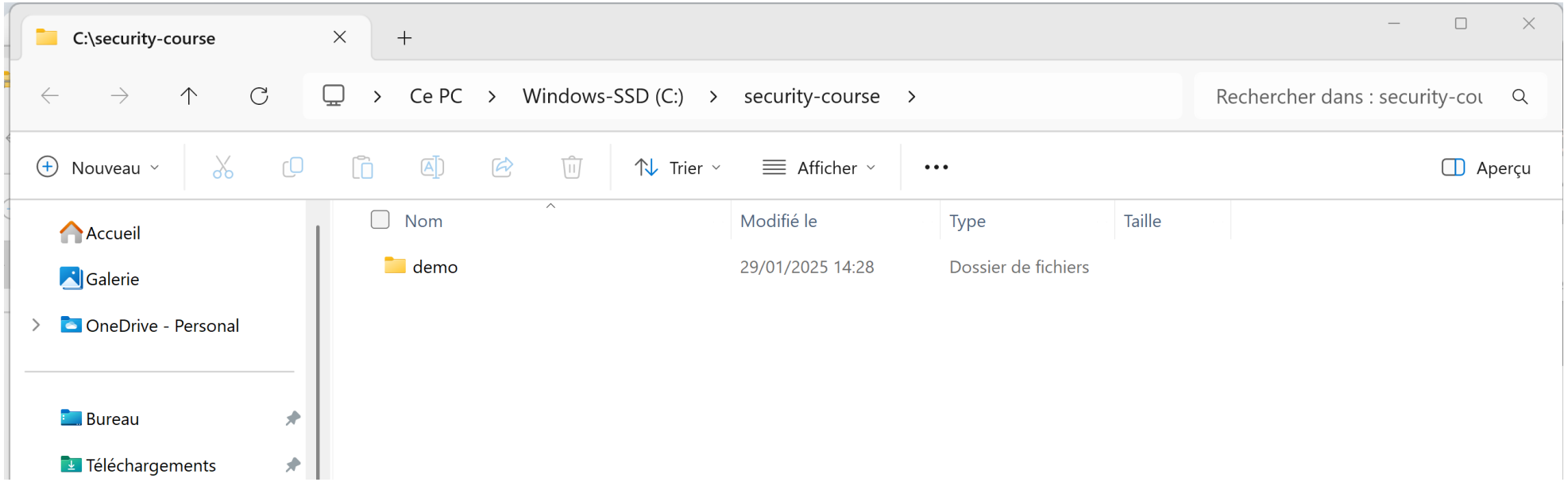


Extract All in a local dir (example in c:\security-course)





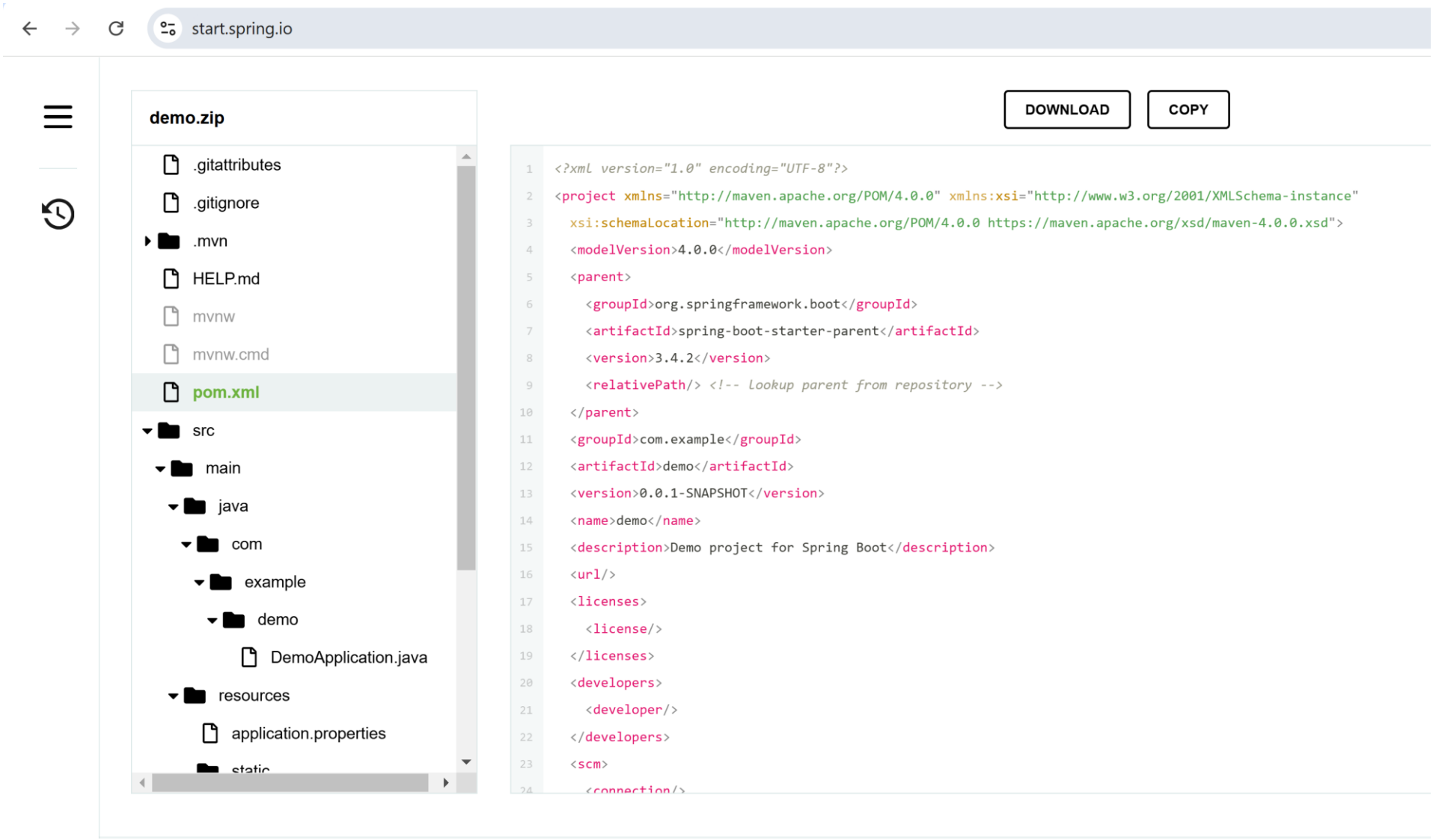
Explore Dir & Files



Basically 2 interresting files:

"pom.xml" => maven description of the java project
"src/main/java/com/example/demo/DemoApplication.java" => the main program (= the web app)

Explore Dir & Files from initializr "EXPLORE"



pom.xml contains basic information
+ springboot dependencies: starter-web

pom.xml (demo)

```
2    <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w  4
23       <scm>
27           <url/>
28       </scm>
29       <properties>
30           <java.version>17</java.version>
31       </properties>
32       <dependencies> Add Starters...
33           <dependency>
34               <groupId>org.springframework.boot</groupId>
35               <artifactId>spring-boot-starter-web</artifactId>
36           </dependency>
37
```

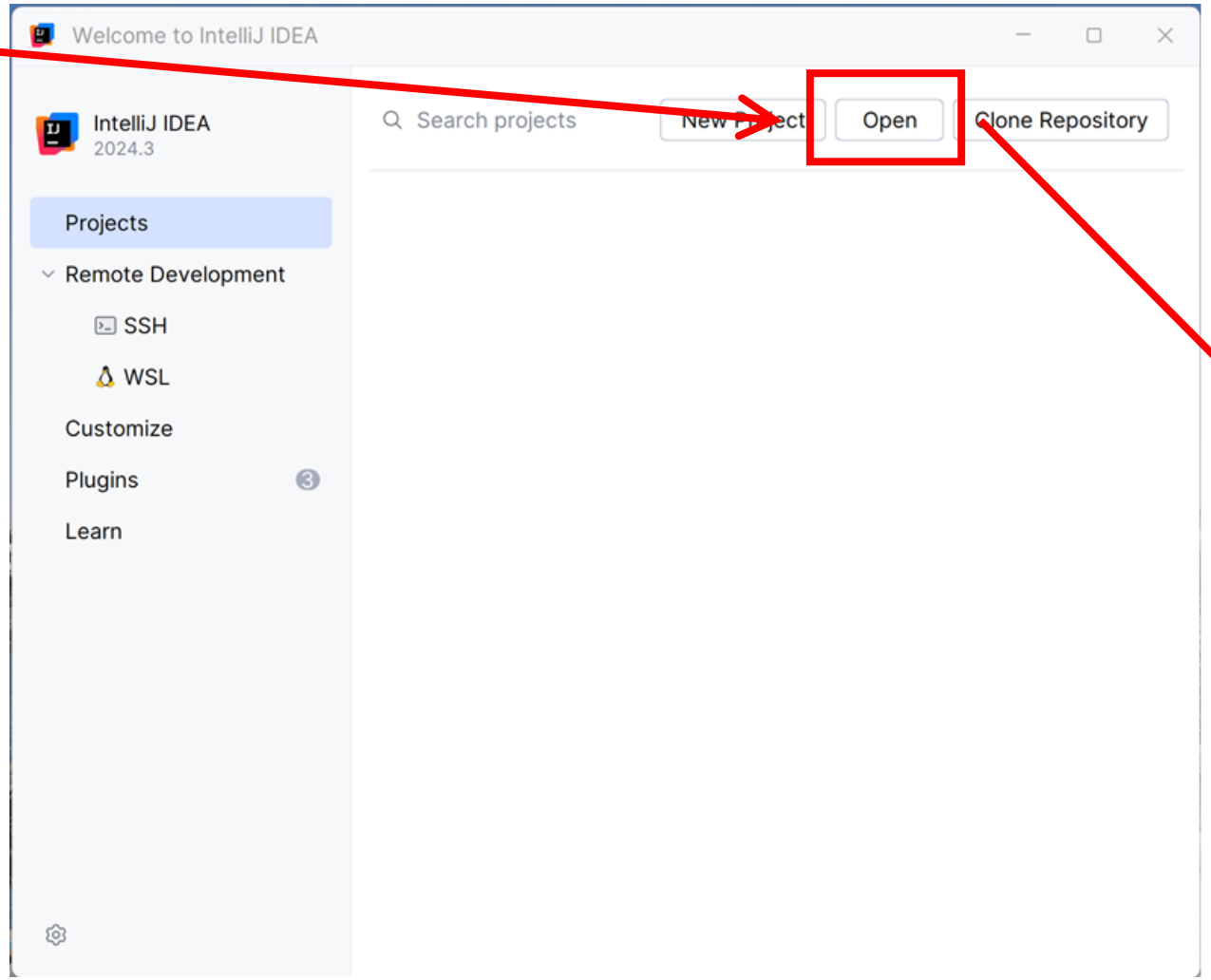
TD Outline

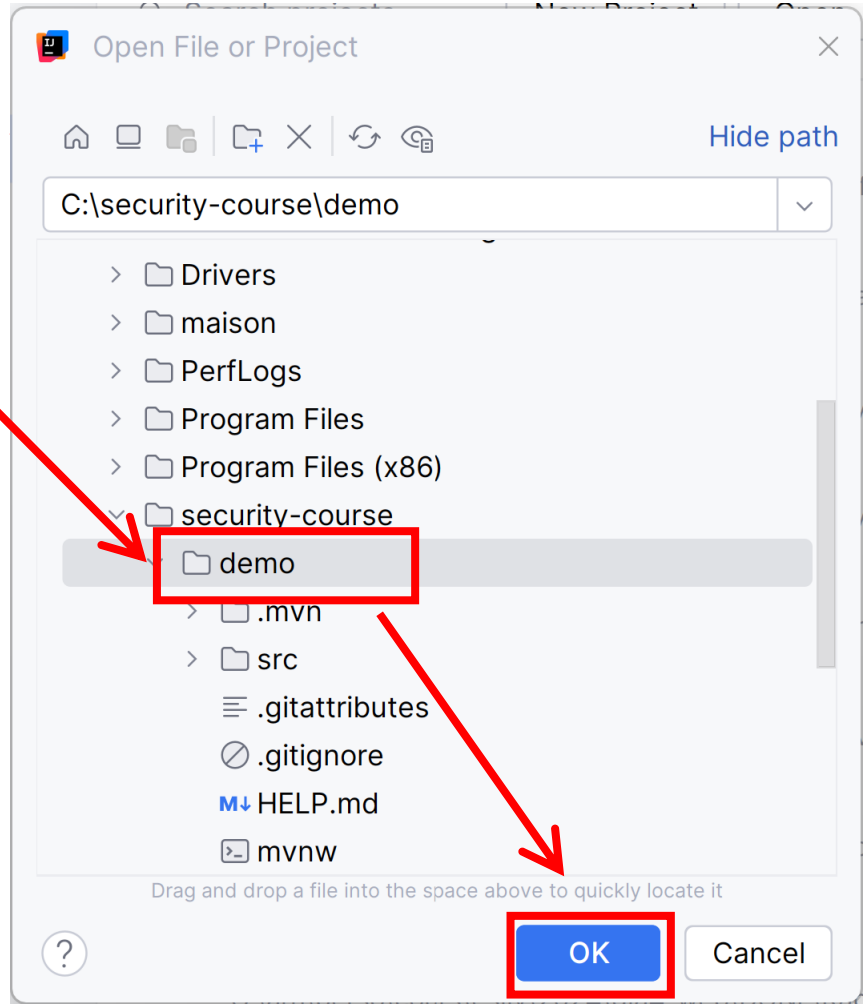
TD Goals = debug yourself a basic Http client-server

1/ install "Java JDK"
2/ install IDE : "IntelliJ IDEA"
3/ setup your app, using spring "initializr" click+download
4/ run your app
5/ open http://localhost:8080
6/ inspect network requests
7/ test Http requests

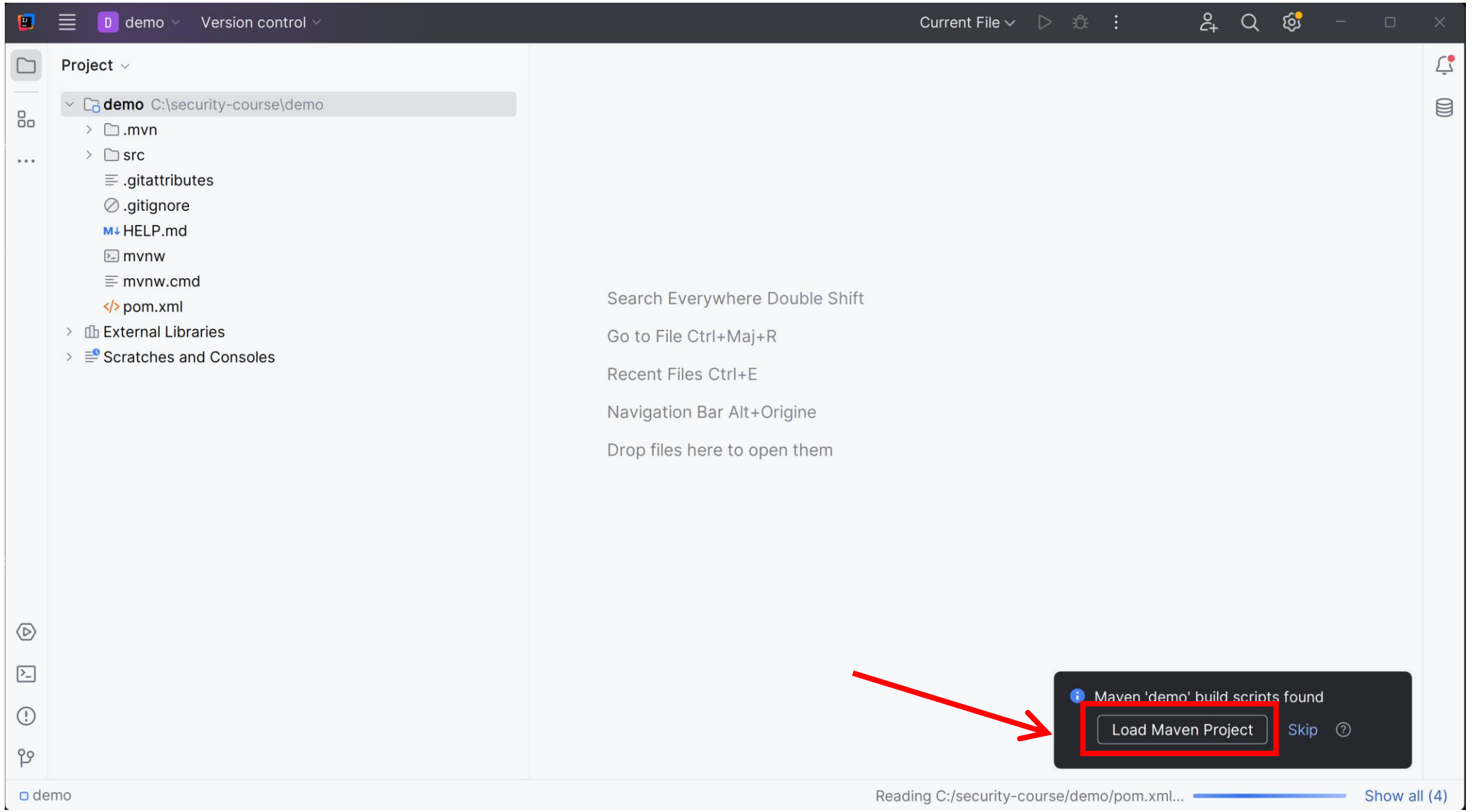
Launch your IDEA IntelliJ Open Project



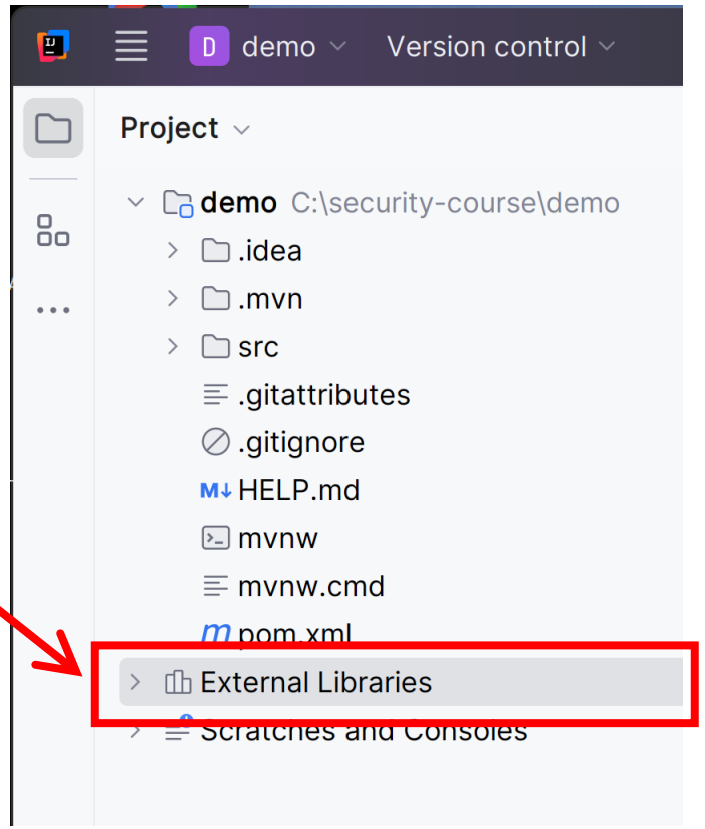




Click default "Load Maven Project"



Wait few minutes (downloading dependencies ...)

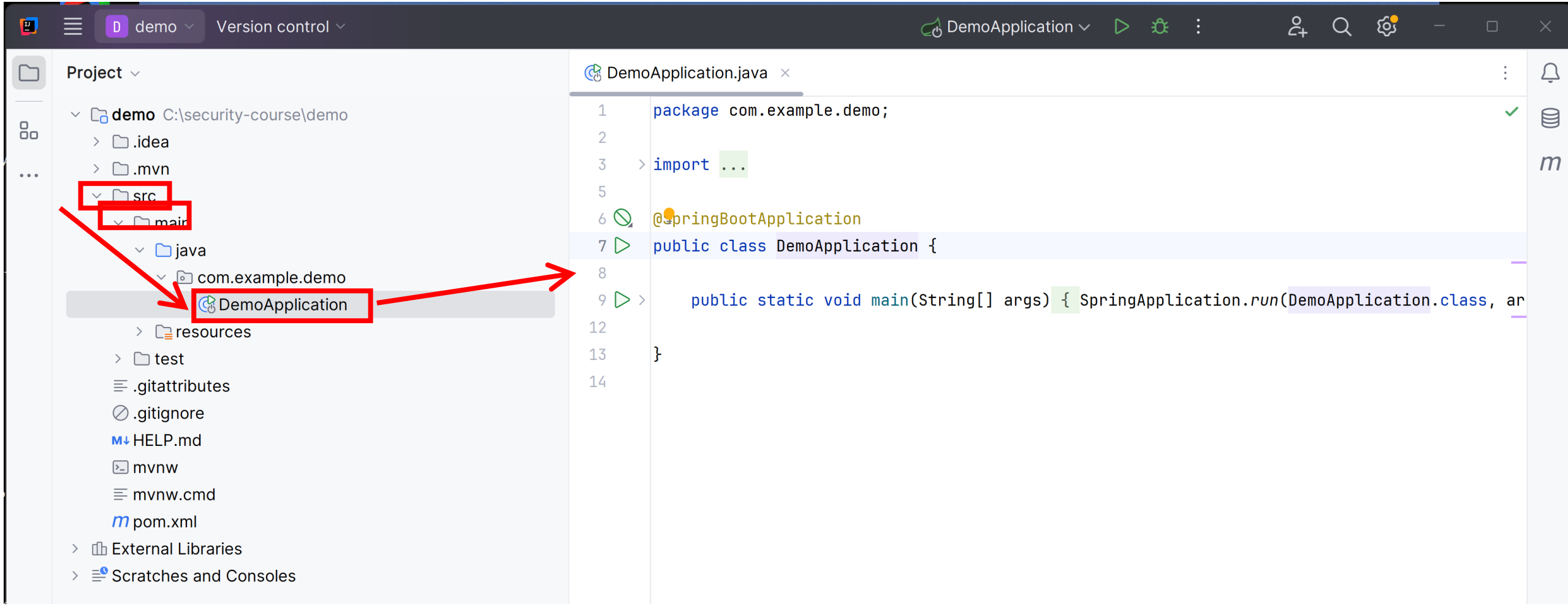


when finished
All dependencies
are automatically
downloaded

spring -> tomcat -> ...



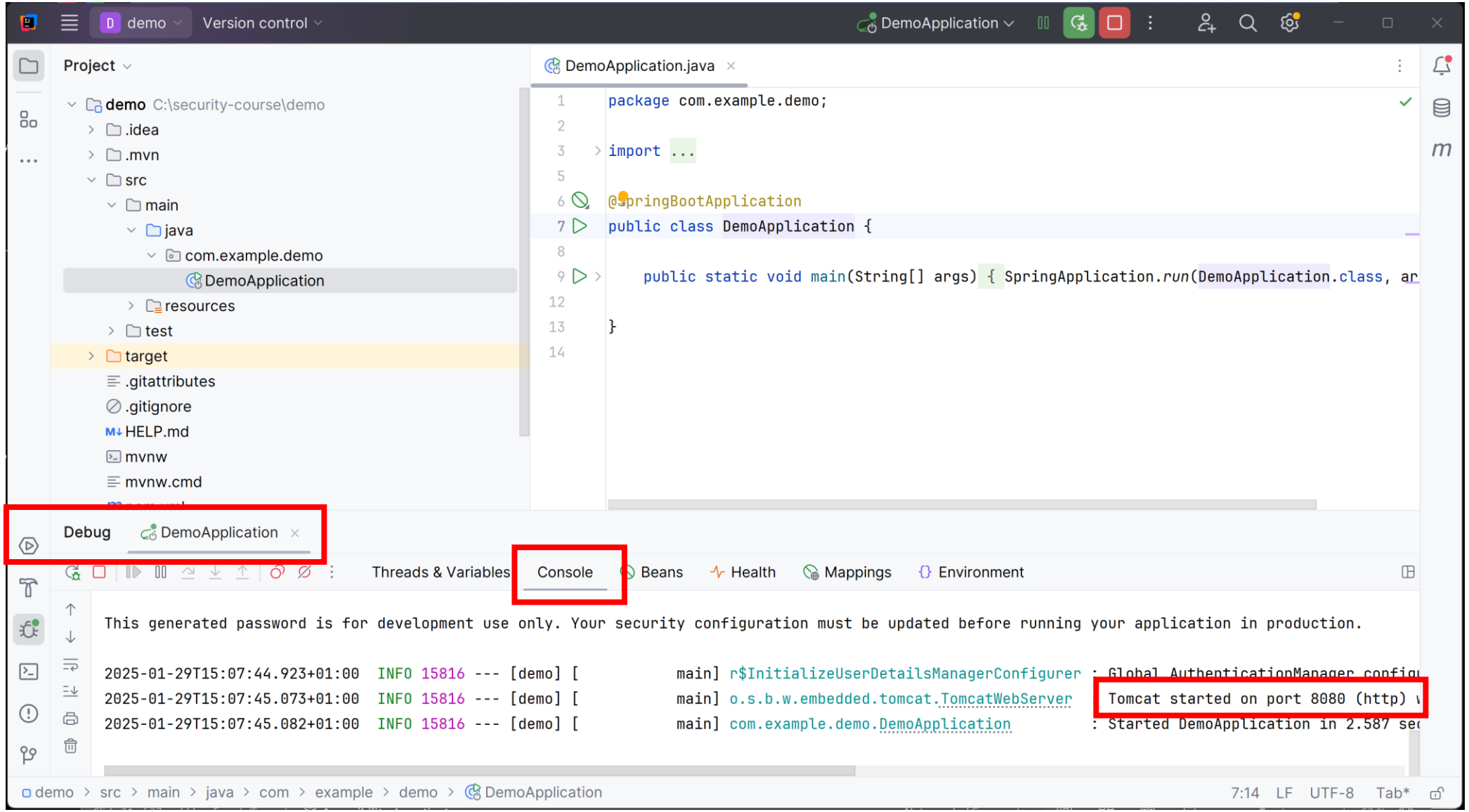
Open the main class "DemoApplication" in src/main/java/com.example.demo/



Click on "Run" Debug button



See Debug "Console" logs...
Tomcat started on port 8080 (http)

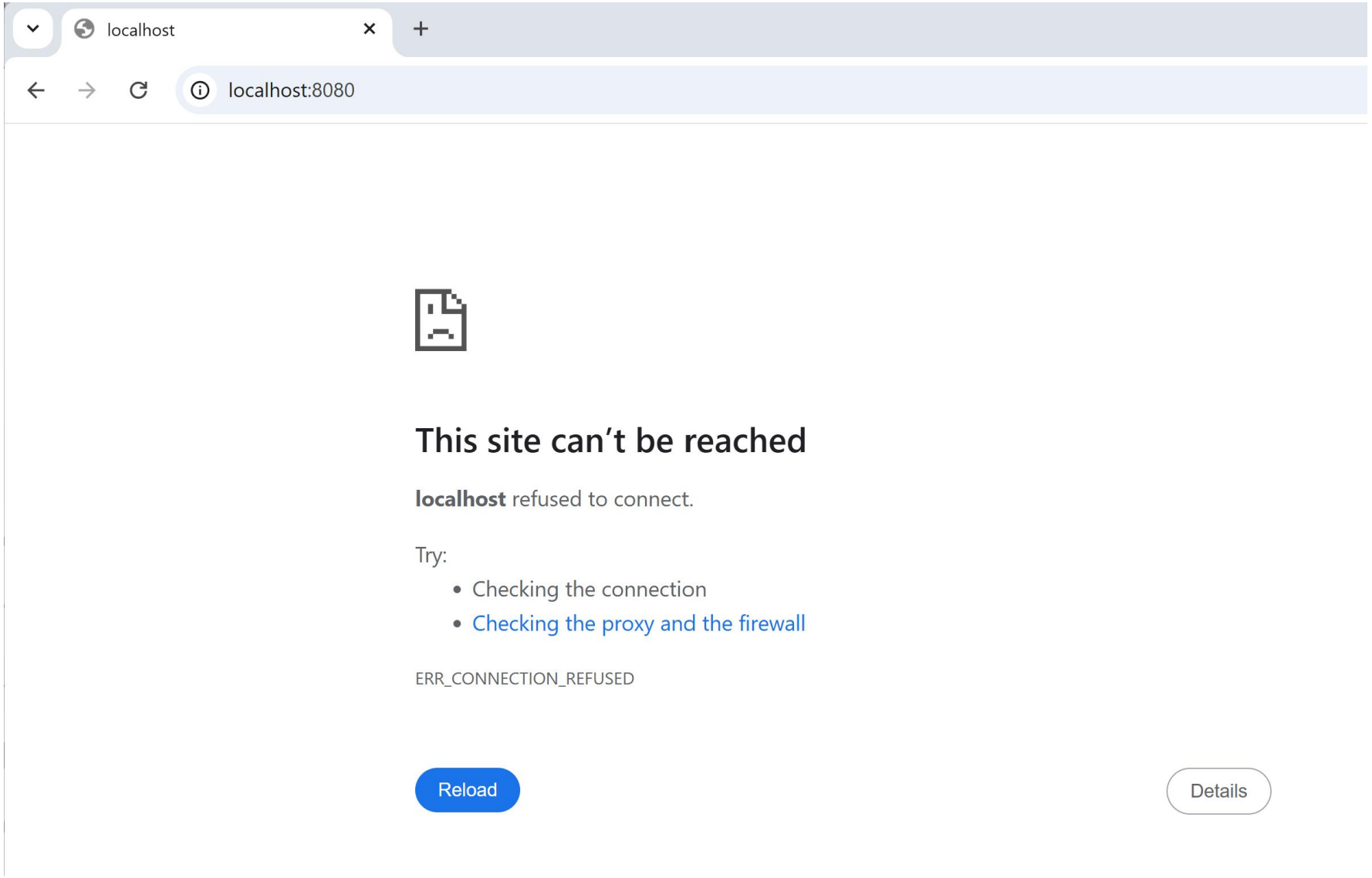


TD Outline

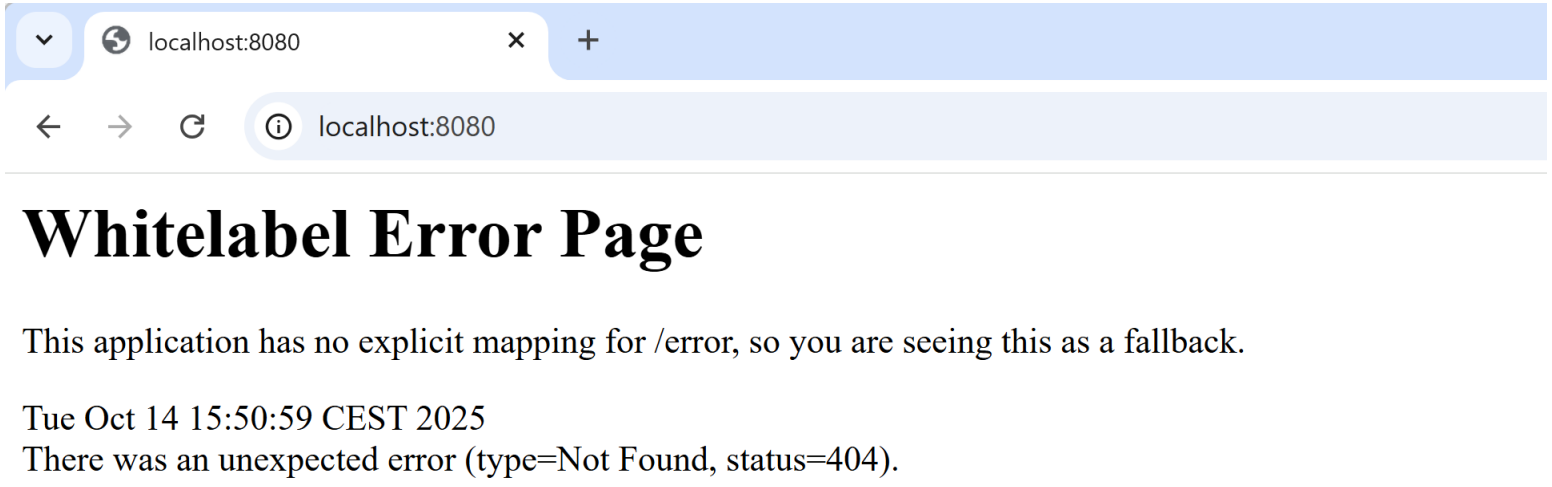
TD Goals = debug yourself a basic Http client-server

1/ install "Java JDK"
2/ install IDE : "IntelliJ IDEA"
3/ setup your app, using spring "initializr" click+download
4/ run your app
5/ open http://localhost:8080
6/ inspect network requests
7/ test Http requests

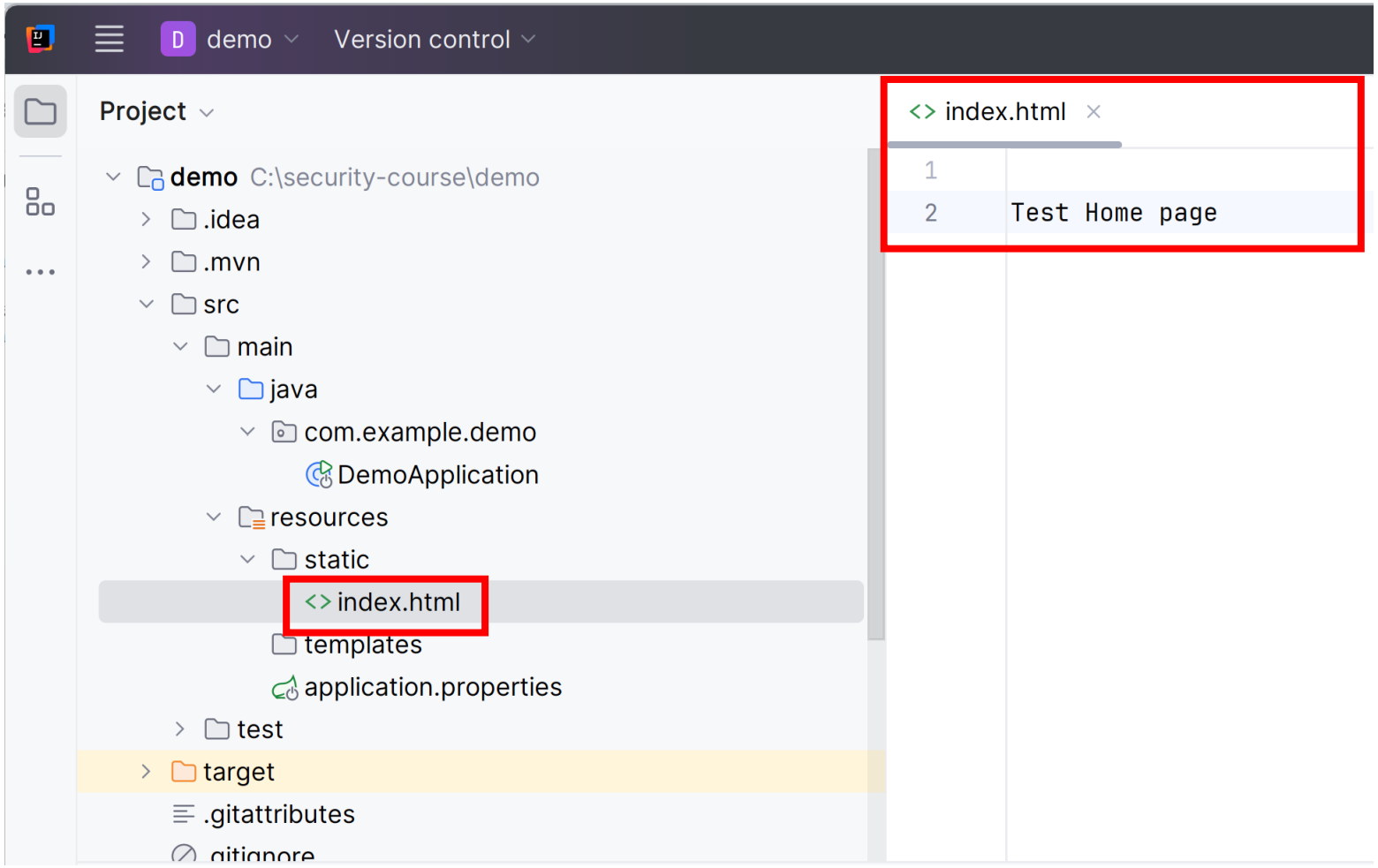
If server not running (or different port)



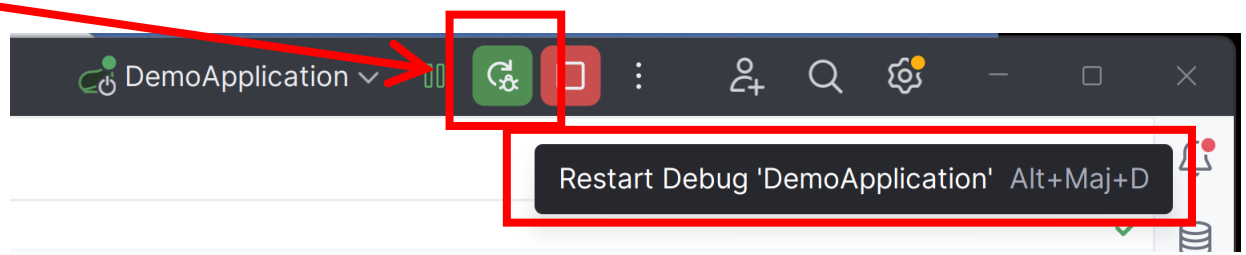
404 … page not found

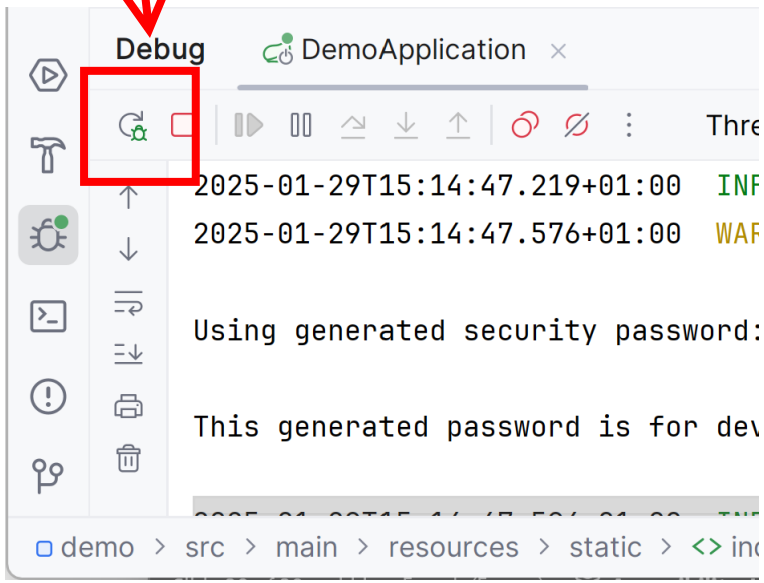


Adding new file "src/main/resource/static/index.html"



Relaunch


DemoApplication
Restart Debug 'DemoApplication' Alt+Maj+D


Debug
DemoApplication
2025-01-29T15:14:47.219+01:00
2025-01-29T15:14:47.576+01:00
Using generated security password:
This generated password is for dev
demo
src
main
resources
static

Retry ...
Resfresh (F5) Web page
login user / (new) generated password



Add binding Java Code for http Requests

Create java class: « TodoRestController » and copy following…

```
package com.example.demo;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.ArrayList;
import java.util.List;

@RestController @RequestMapping(path = "/api/todo")
public class TodoRestController {

   public static class Todo {
      public int id; public String title; public String description; public int priority;

      public Todo() { }
      public Todo(int id, String title, String description, int priority) {
         this.id = id;      this.title = title;    this.description = description;
         this.priority = priority;
      }
   }

   private int idGenerator = 1;
   private int newId() {
      return idGenerator++;
   }
   protected List<Todo> todos = new ArrayList<>(List.of(
         new Todo(newId(), "Learn Java", "", 1),
         new Todo(newId(), "Learn SpringBoot", "", 2),
         new Todo(newId(), "Learn Rest Spring-Web", "", 3)
   ));

// READ - Get all todos
   @GetMapping
   public List<Todo> getAll() {
      return todos;
   }

   // READ - Get todo by ID
   @GetMapping("/{id}")
   public ResponseEntity<Todo> getById(@PathVariable int id) {
      Todo todo = todos.stream().filter(t -> t.id == id).findFirst().orElse(null);
      if (todo == null) {
         return ResponseEntity.notFound().build();
      }
      return ResponseEntity.ok(todo);
   }

   // CREATE - Add new todo
   @PostMapping
   public ResponseEntity<Todo> create(@RequestBody Todo todo) {
      if (todo == null) {
         return ResponseEntity.badRequest().build();
      }
      todo.id = newId(); // Assign new ID
      todos.add(todo);
      return ResponseEntity.status(HttpStatus.CREATED).body(todo);
   }

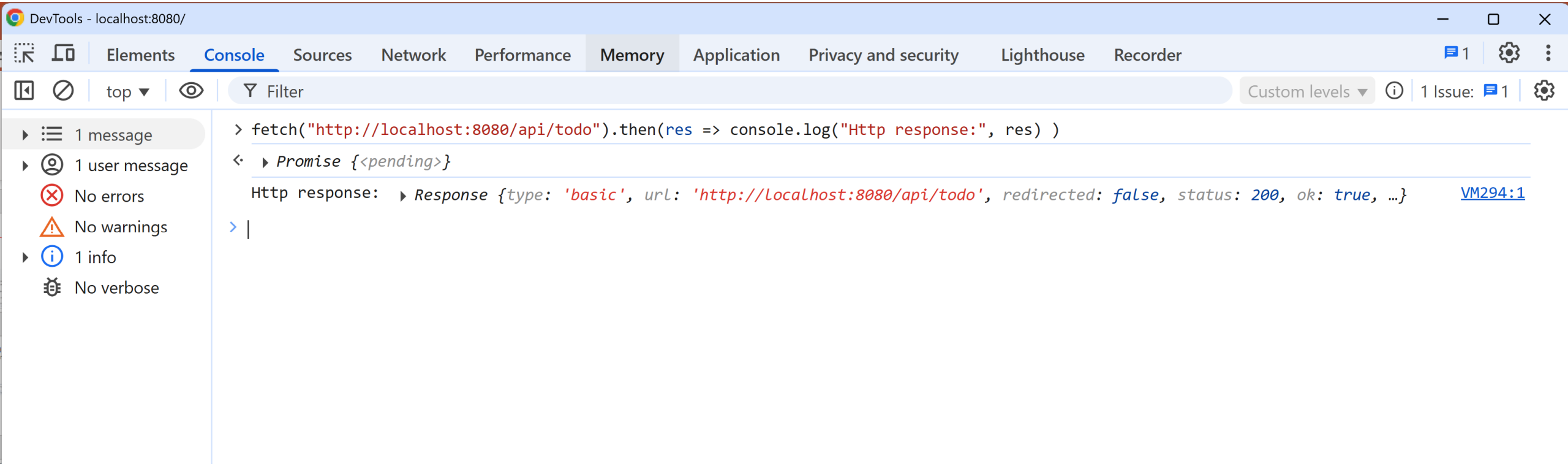
   // UPDATE - Update existing todo
   @PutMapping()
   public ResponseEntity<Todo> update(
         @RequestBody Todo updatedTodo) {
      if (updatedTodo == null) {
         return ResponseEntity.badRequest().build();
      }
      int id = updatedTodo.id;
      boolean removed = todos.removeIf(t -> t.id == id);
      if (!removed) {
         return ResponseEntity.notFound().build();
      }
      todos.add(updatedTodo);
      return ResponseEntity.ok(updatedTodo);
   }

   // DELETE - Delete todo by ID
   @DeleteMapping("/{id}")
   public ResponseEntity<Void> delete(@PathVariable int id) {
      boolean removed = todos.removeIf(t -> t.id == id);
      if (removed) {
         return ResponseEntity.noContent().build();
      }
      return ResponseEntity.notFound().build();
   }

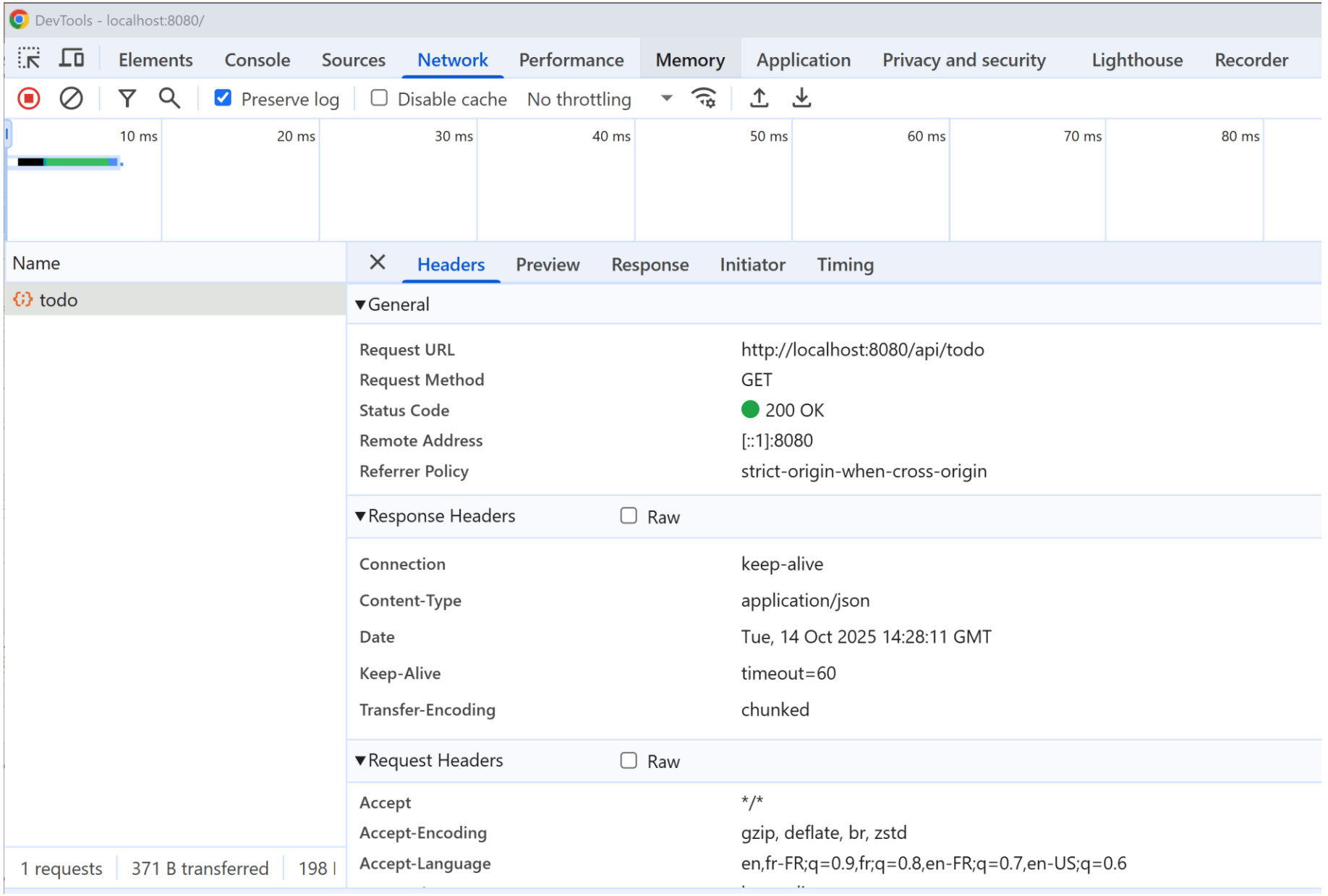
   // DELETE - Delete all todos
   @DeleteMapping
   public ResponseEntity<Void> deleteAll() {
      todos.clear();
      return ResponseEntity.noContent().build();
   }
}
```

Test Using Web Browser DevTool « fetch() »

Open browser page « http://localhost:8080/ »
Then Open DevTool « F12 »
Then go in « Console » TAB
Type fetch("http://localhost:8080/api/todo").then(res => console.log("Http response:", res))

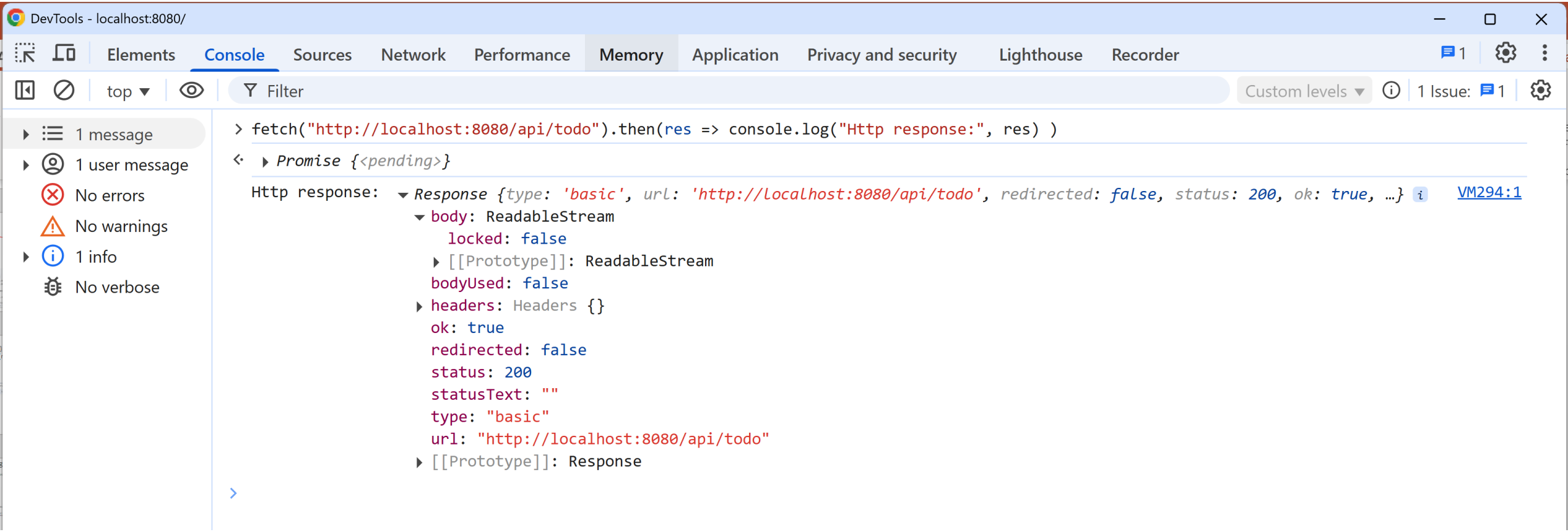


Open DevTools Tab « Network »



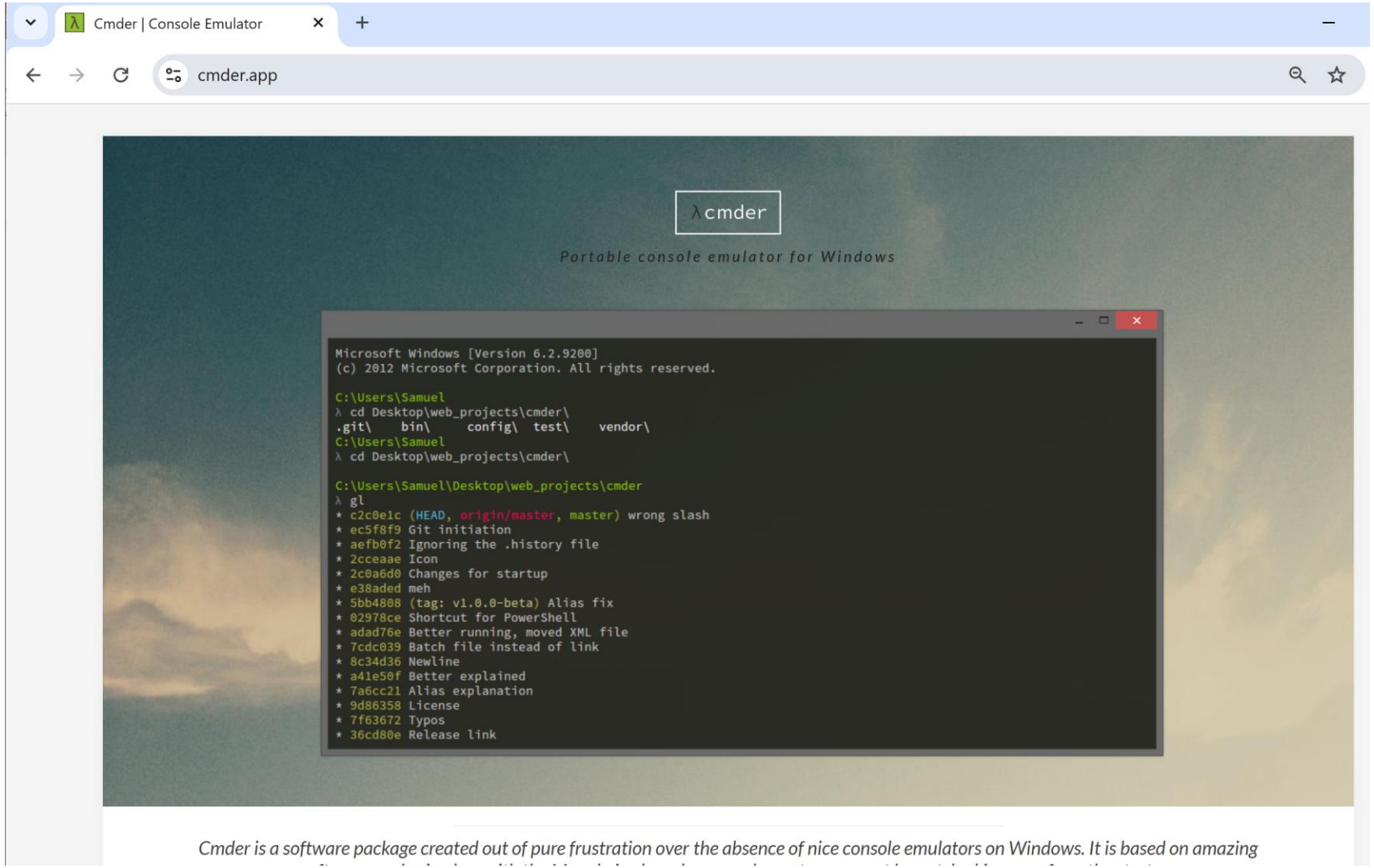
DevTool Console LOG INFO

Difficult to read the « Response Body » (ReadableStream)

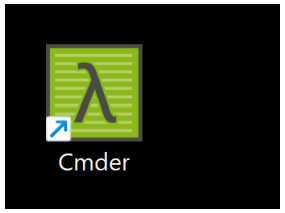


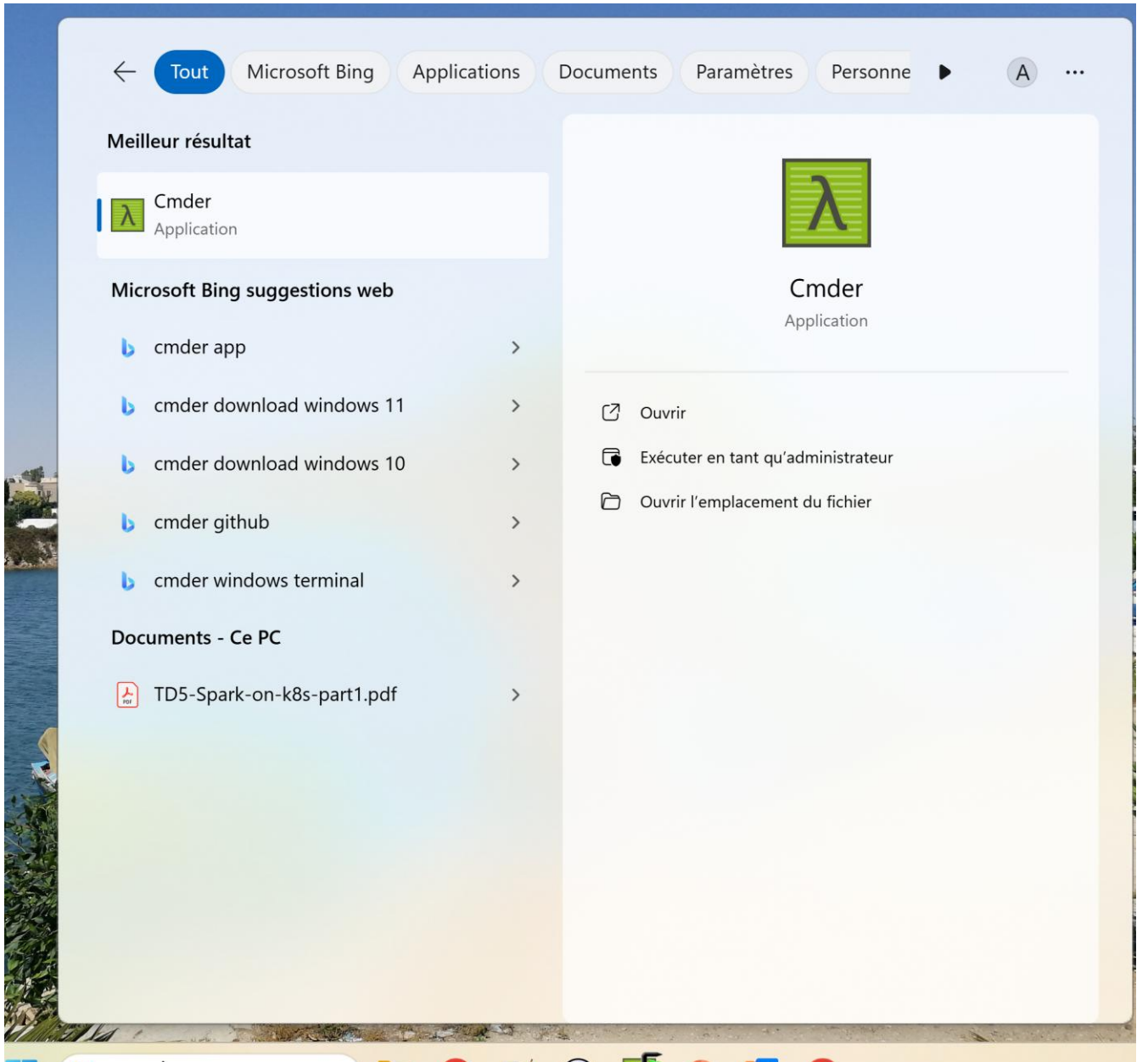
Testing Using CURL … need install Cmder

https://cmder.app

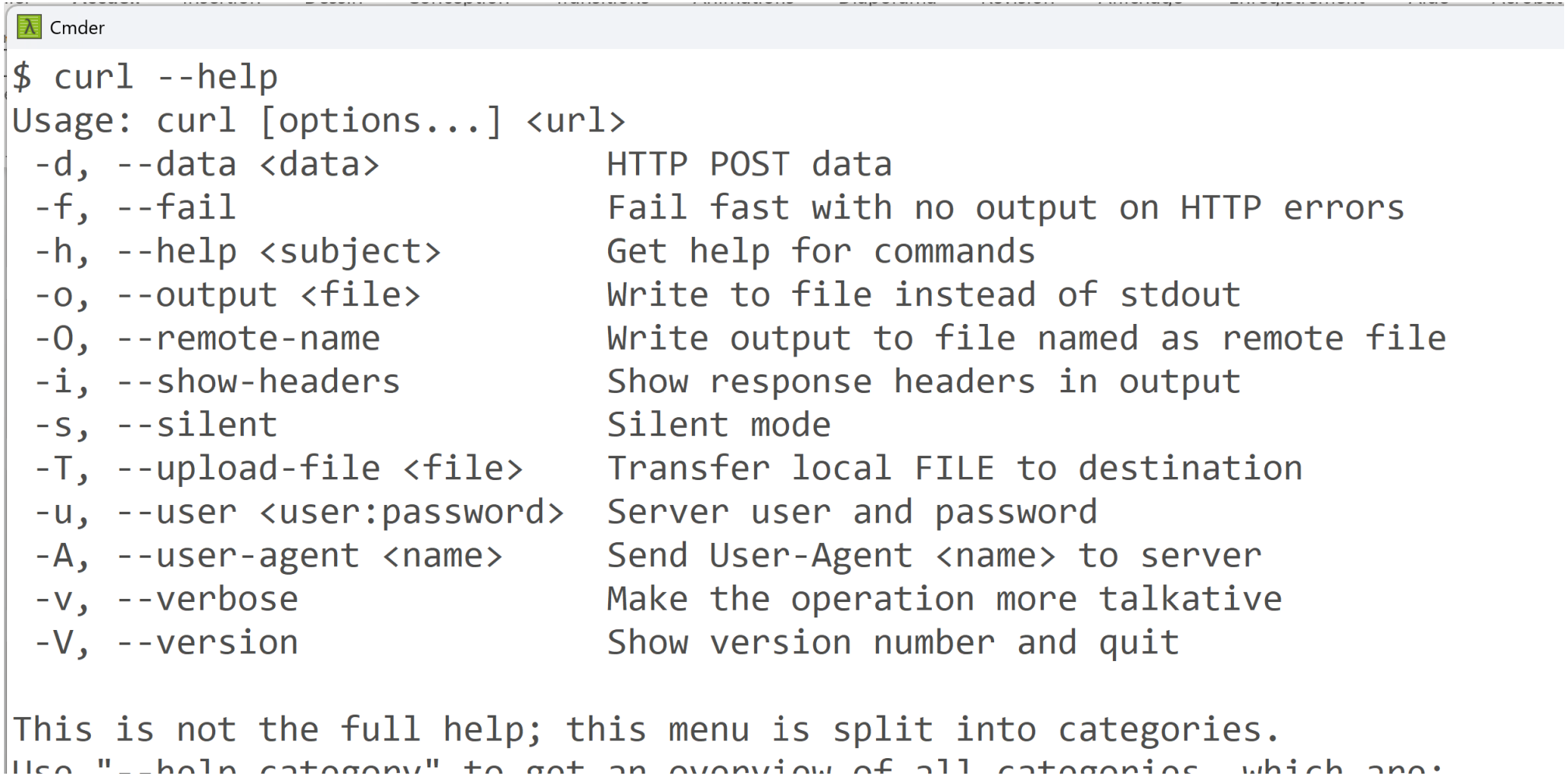


Open Terminal for Cmder

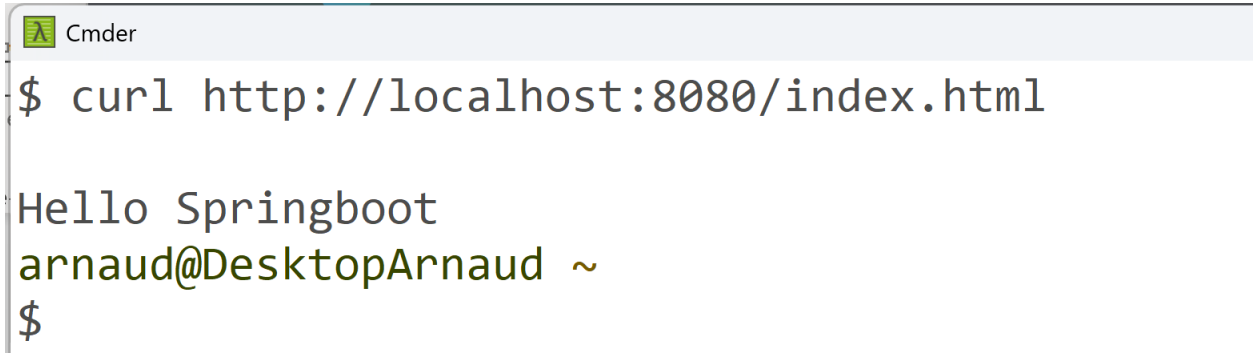




$ curl command



Example curl GET



Curl Requests

```
# GET
curl http://localhost:8080/api/todo

curl http://localhost:8080/api/todo/2

# POST
curl -X POST http://localhost:8080/api/todo -d '{"description":"Test"}' \
            -H "Content: application/json" -H "Accept: application/json"

# PUT
curl -X PUT http://localhost:8080/api/todo -d '{"id":4, "description":"New Test"}' \
            -H "Content: application/json" -H "Accept: application/json"

# DELETE
curl -X DELETE http://localhost:8080/api/todo/2
             -H "Accept: application/json"
```

Using Curl, with Headers « -H ‘key: value’ »

Cmder

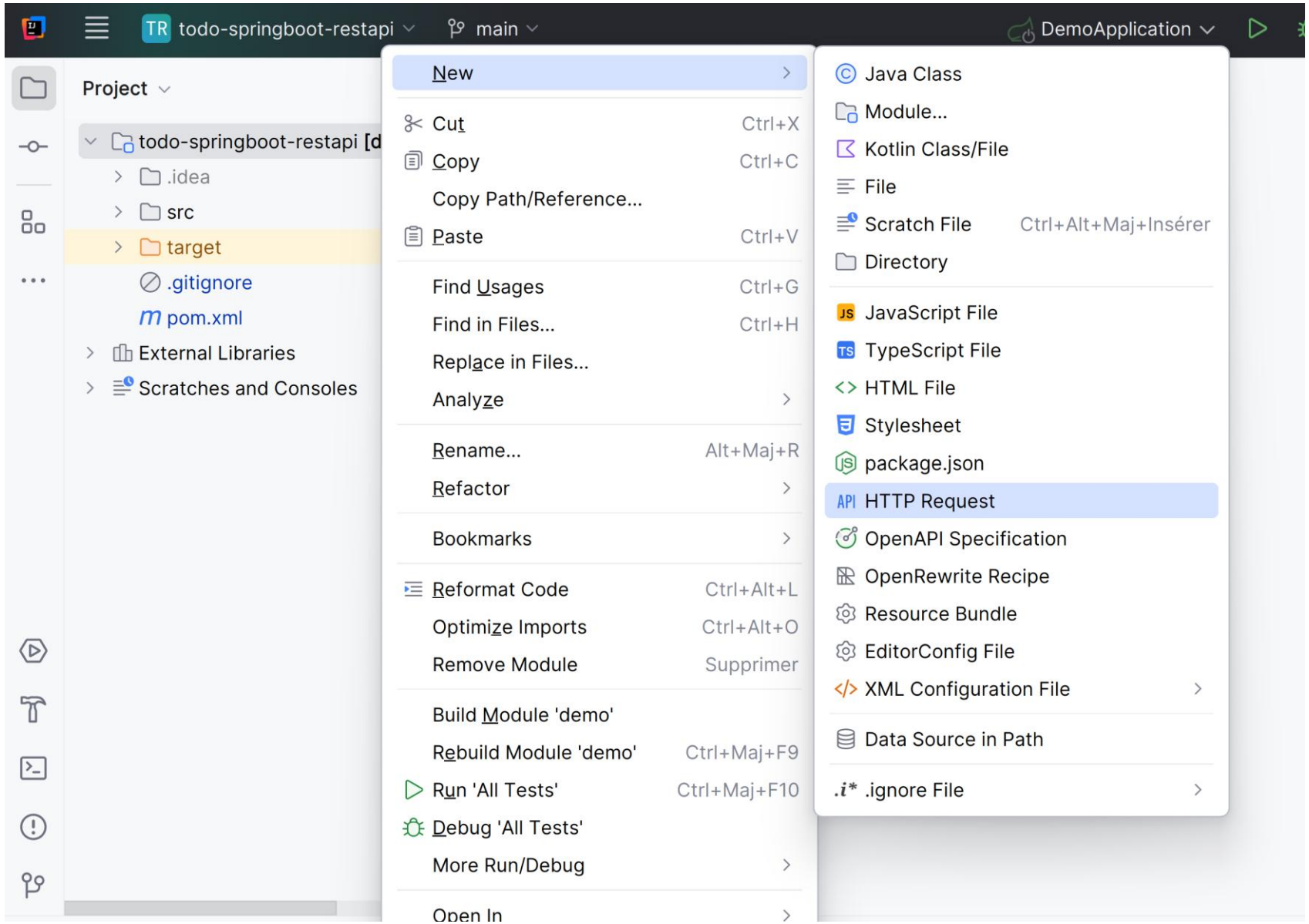
```
$ curl -v -H "accept: application/json" http://localhost:8080/api/todo
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
* using HTTP/1.x
> GET /api/todo HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.14.1
> accept: application/json
>
* Request completely sent off
< HTTP/1.1 200
< Content-Type: application/json
< Transfer-Encoding: chunked
< Date: Tue, 14 Oct 2025 14:02:09 GMT
<
[{"id":1,"title":"Learn Java","description":"","priority":1},{"id":2,"title":"Learn SpringBoot","description":""
,"priority":2},{"id":3,"title":"Learn Rest Spring-Web","description":"","priority":3}]* Connection #0 to host lo
calhost left intact
```

Example http 406 (bad media)

Cmder

```
arnaud@DesktopArnaud ~
$ curl -v -H "accept: application/xml" http://localhost:8080/api/todo
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
* using HTTP/1.x
> GET /api/todo HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.14.1
> accept: application/xml
>
* Request completely sent off
< HTTP/1.1 406
< Accept: application/json, application/*+json
< Content-Length: 0
< Date: Tue, 14 Oct 2025 14:00:42 GMT
<
* Connection #0 to host localhost left intact
```

Test Using IntelliJ « test.http » file



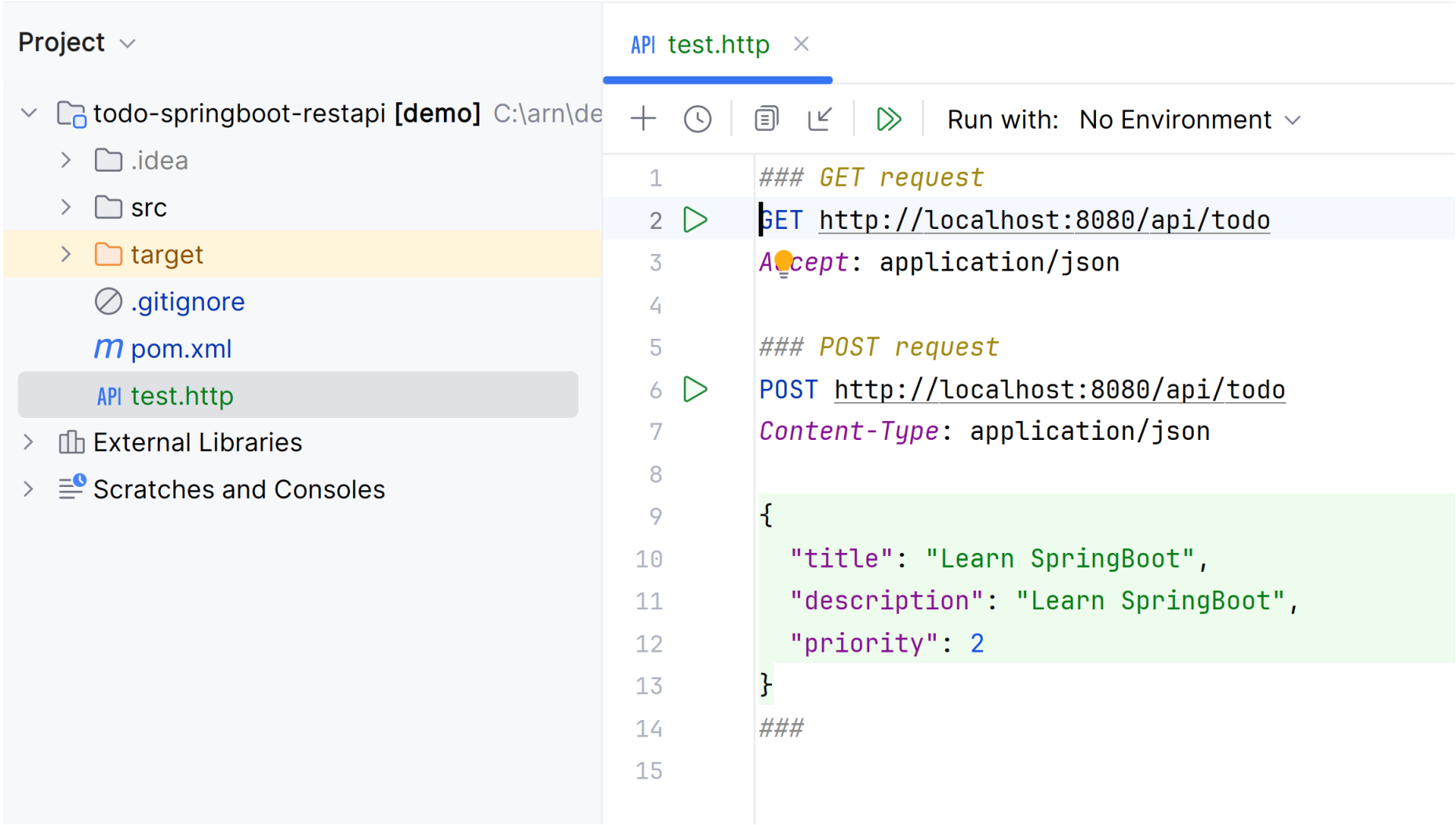
Edit « test.http » file

```
### GET request
GET http://localhost:8080/api/todo
Accept: application/json

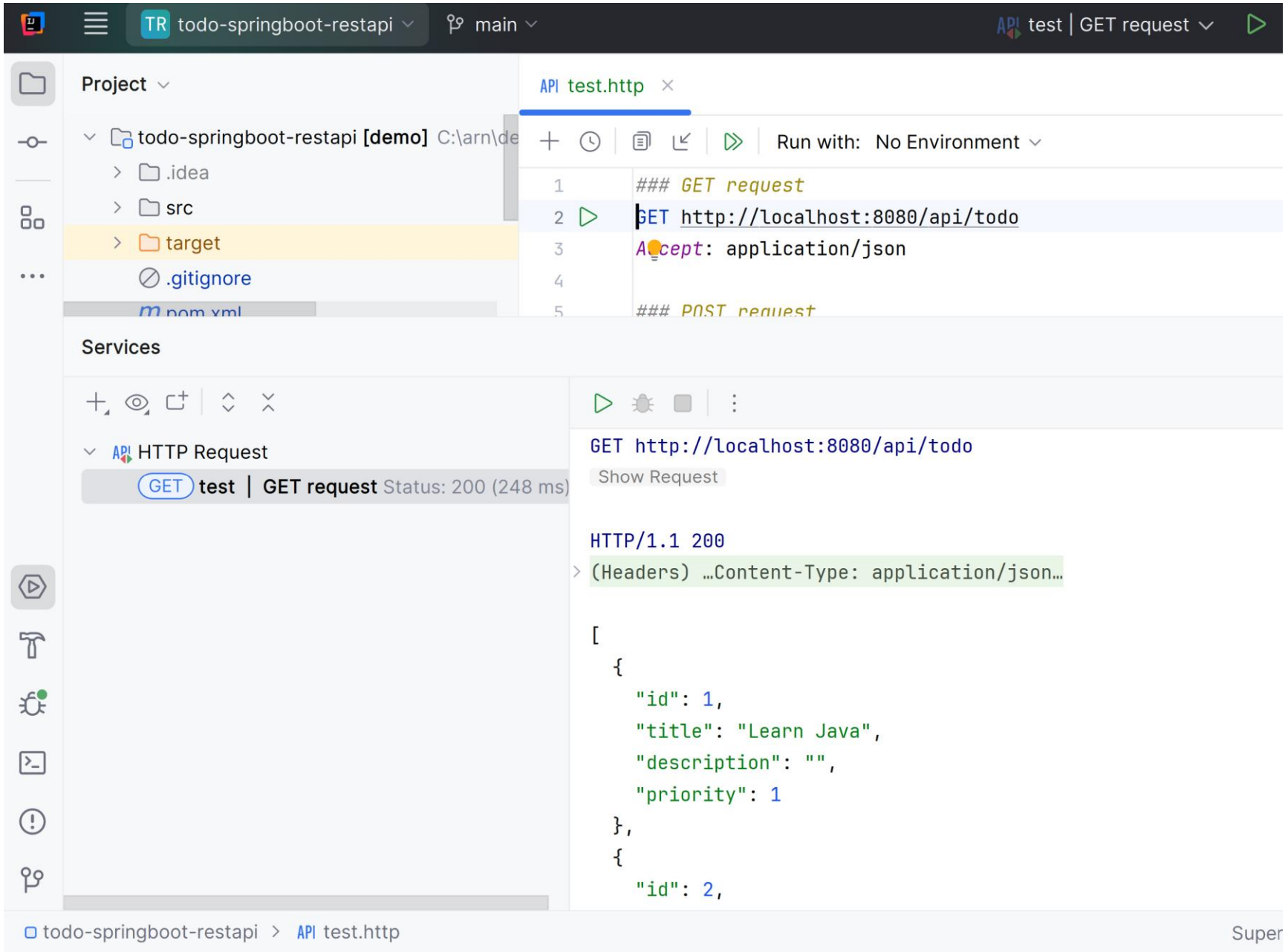
### POST request
POST http://localhost:8080/api/todo
Content-Type: application/json

{
  "title": "Learn SpringBoot",
  "description": "Learn SpringBoot",
  "priority": 2
}
###
```

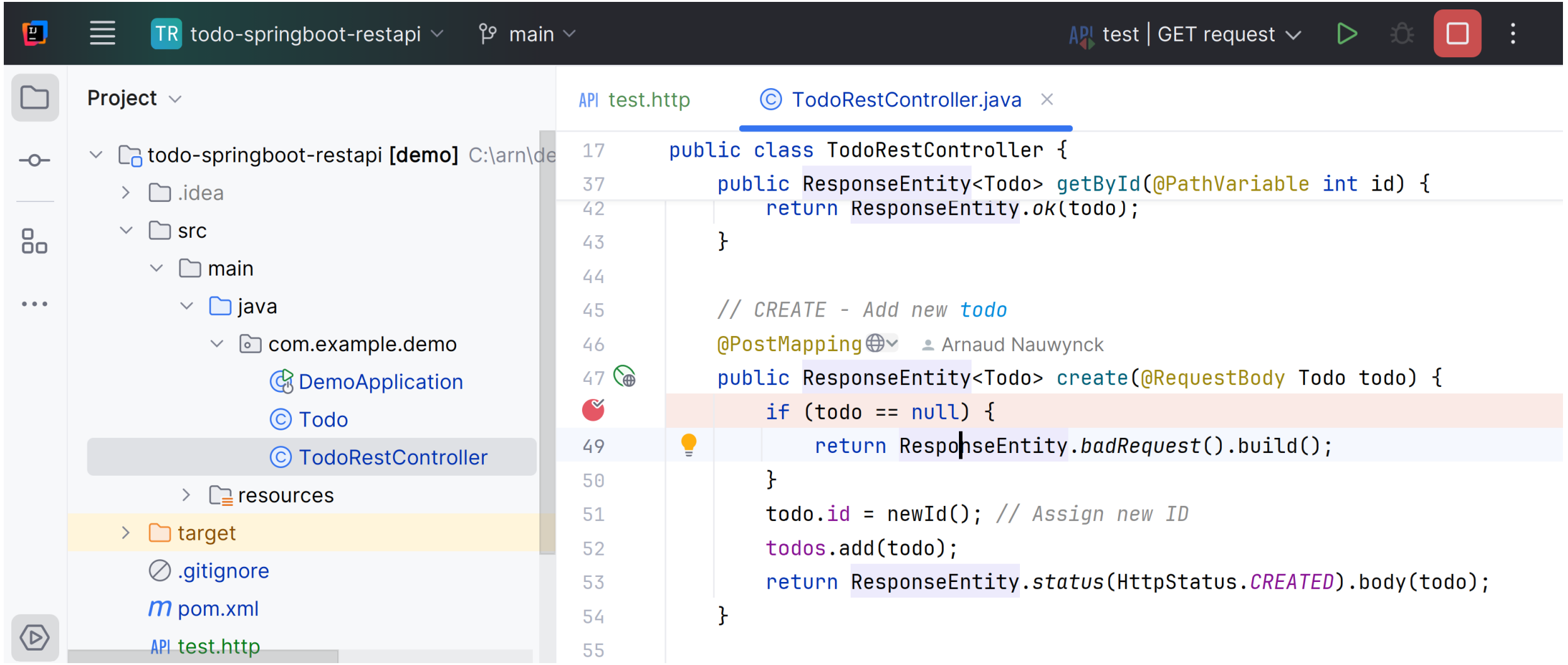
Test.http ... request are executable



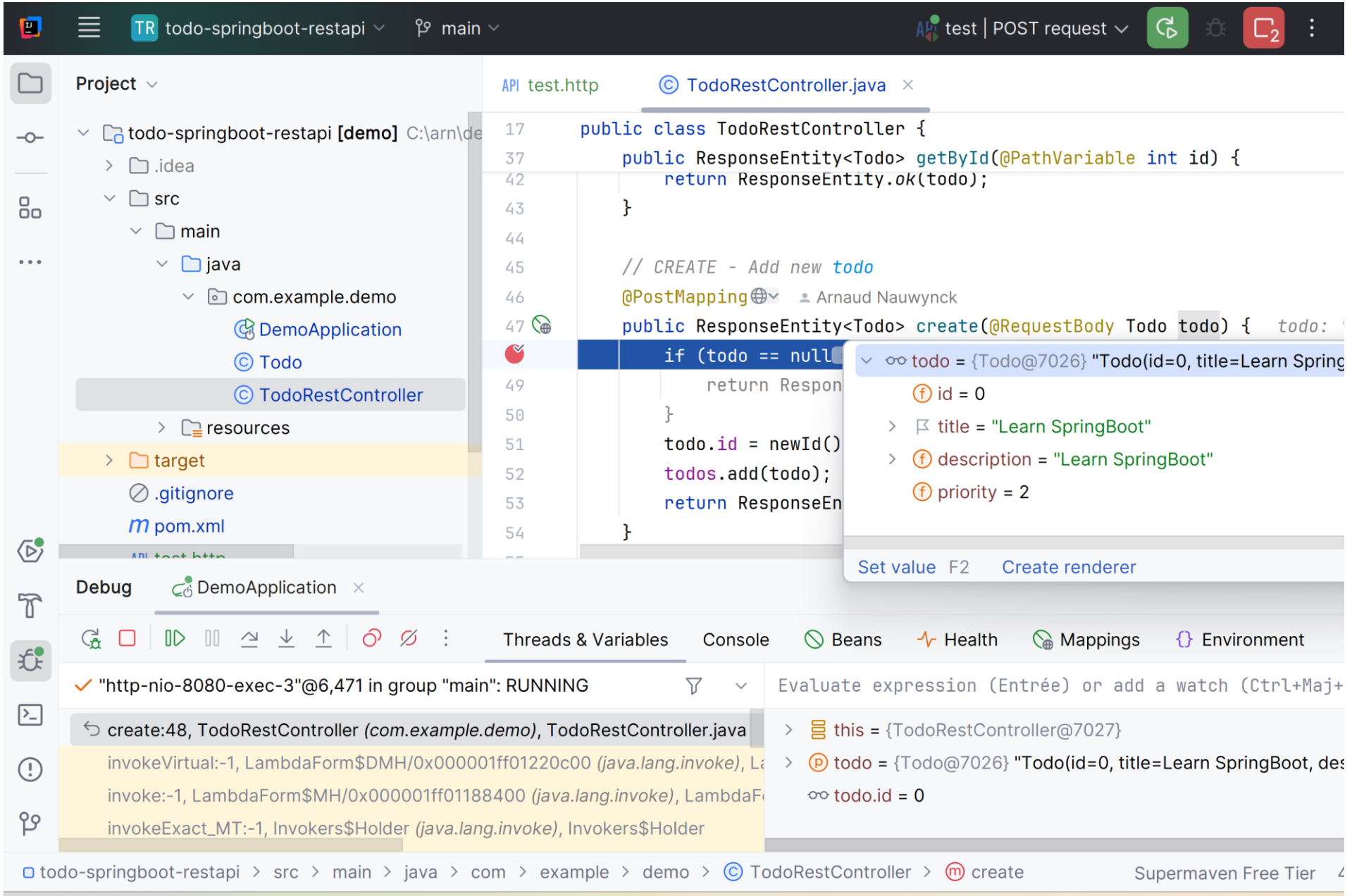
Execute..


TR todo-springboot-restapi
main
test | GET request
Project
todo-springboot-restapi [demo] C:\arn\de
.idea
src
target
.gitignore
pom.xml
API test.http
Run with: No Environment
1 ### GET request
2 GET http://localhost:8080/api/todo
3 Accept: application/json
4
5 ### POST request
Services
HTTP Request
GET test | GET request Status: 200 (248 ms)
GET http://localhost:8080/api/todo
Show Request
HTTP/1.1 200
(Headers) …Content-Type: application/json…
[
{
"id": 1,
"title": "Learn Java",
"description": "",
"priority": 1
},
{
"id": 2,
todo-springboot-restapi > API test.http

Put Debug Breakpoints in Server

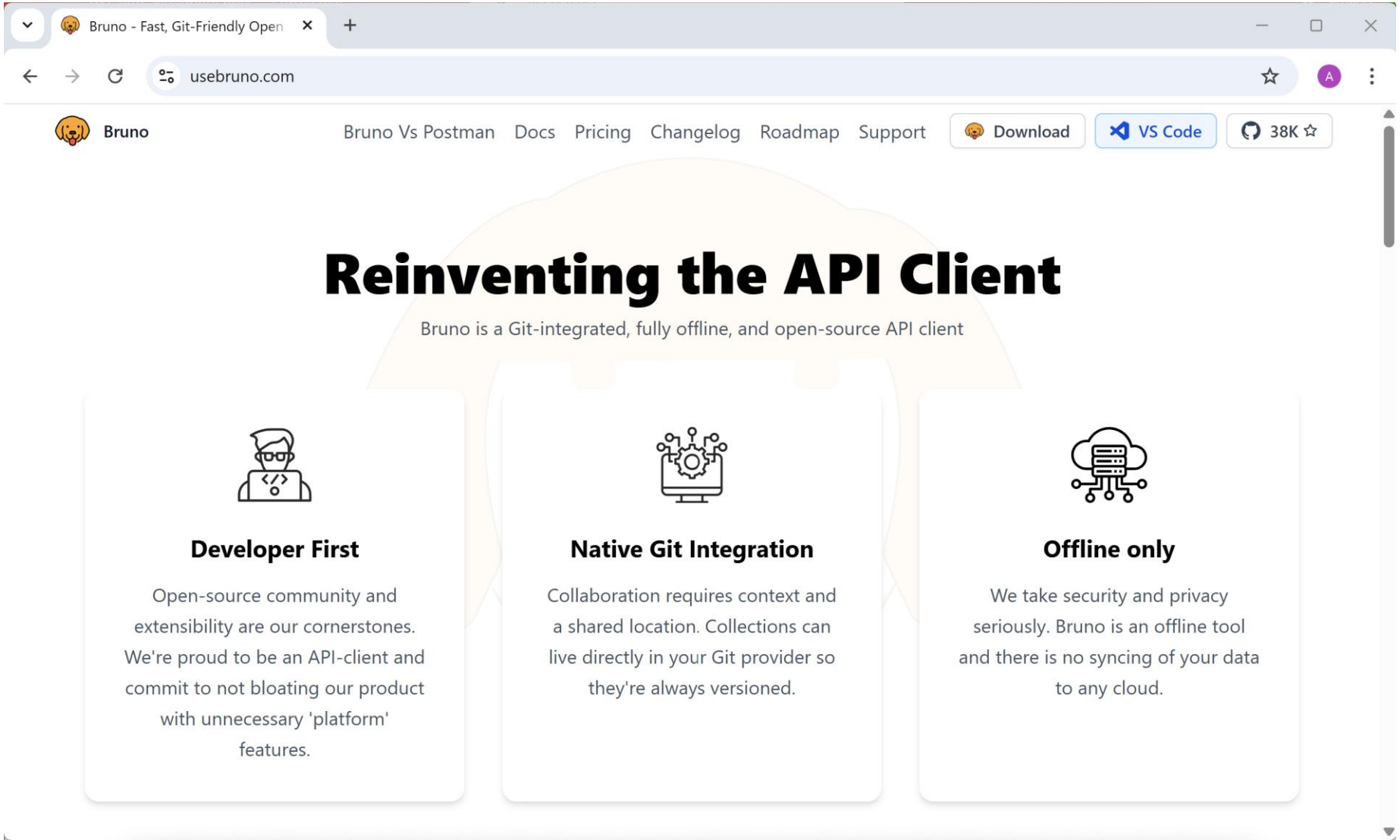


Debug..

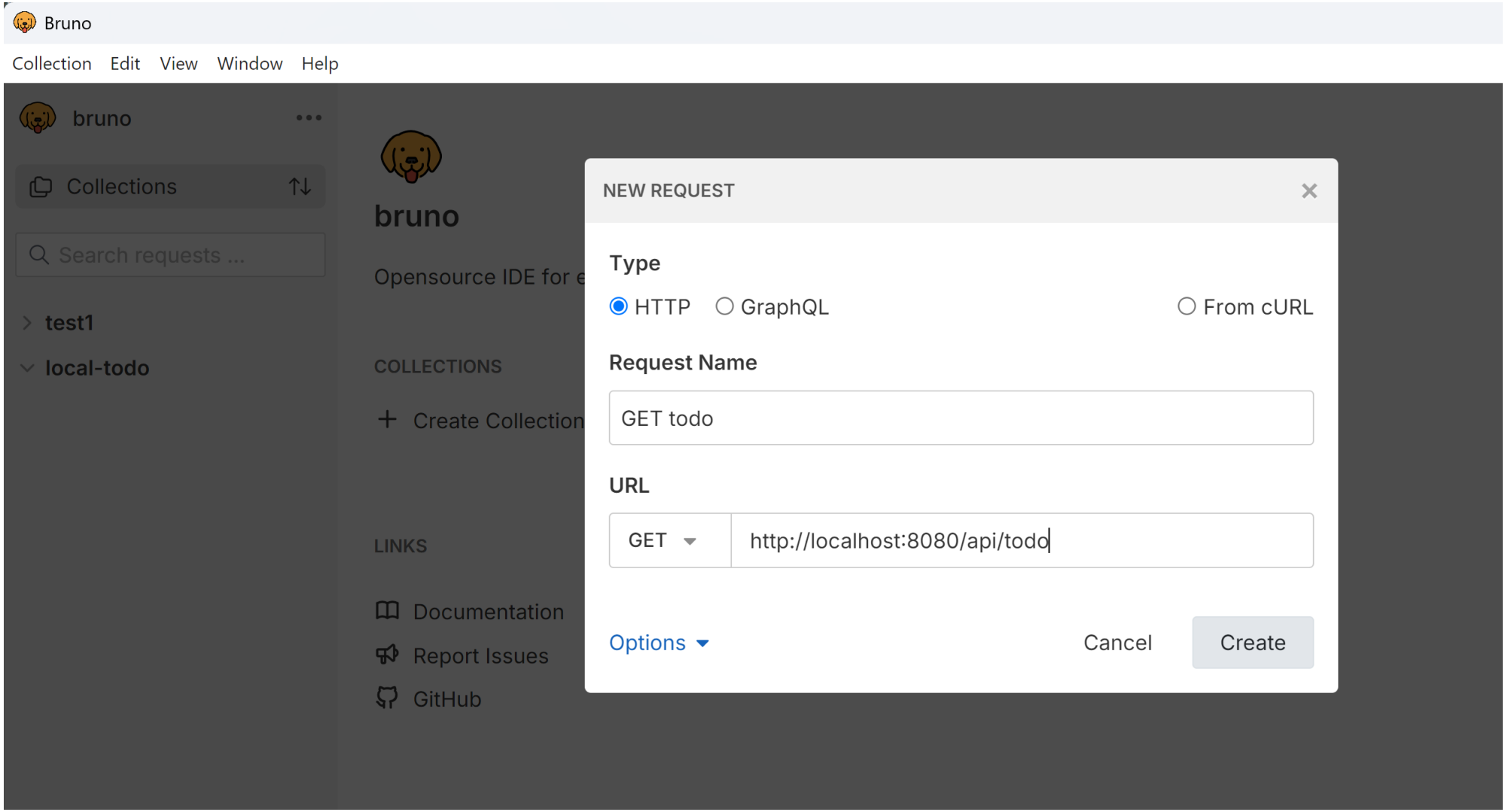

todo-springboot-restapi
main
test | POST request
Project
todo-springboot-restapi [demo] C:\arn\de
.idea
src
main
java
com.example.demo
DemoApplication
Todo
TodoRestController
resources
target
.gitignore
pom.xml
test.http
TodoRestController.java
17 public class TodoRestController {
37 public ResponseEntity<Todo> getById(@PathVariable int id) {
42 return ResponseEntity.ok(todo);
43 }
44
45 // CREATE - Add new todo
46 @PostMapping
Arnaud Nauwynck
47 public ResponseEntity<Todo> create(@RequestBody Todo todo) {
if (todo == null
return Respon
}
todo.id = newId()
todos.add(todo);
return ResponseEn
}
todo = {Todo@7026} "Todo(id=0, title=Learn Spring
id = 0
title = "Learn SpringBoot"
description = "Learn SpringBoot"
priority = 2
Set value F2
Create renderer
Debug
DemoApplication
Threads & Variables
Console
Beans
Health
Mappings
Environment
"http-nio-8080-exec-3"@6,471 in group "main": RUNNING
Evaluate expression (Entrée) or add a watch (Ctrl+Maj+
create:48, TodoRestController (com.example.demo), TodoRestController.java
invokeVirtual:-1, LambdaForm$DMH/0x000001ff01220c00 (java.lang.invoke), La
invoke:-1, LambdaForm$MH/0x000001ff01188400 (java.lang.invoke), LambdaF
invokeExact_MT:-1, Invokers$Holder (java.lang.invoke), Invokers$Holder
this = {TodoRestController@7027}
todo = {Todo@7026} "Todo(id=0, title=Learn SpringBoot, des
todo.id = 0
todo-springboot-restapi > src > main > java > com > example > demo > TodoRestController > create
Supermaven Free Tier

Test Using Bruno (or Postman)

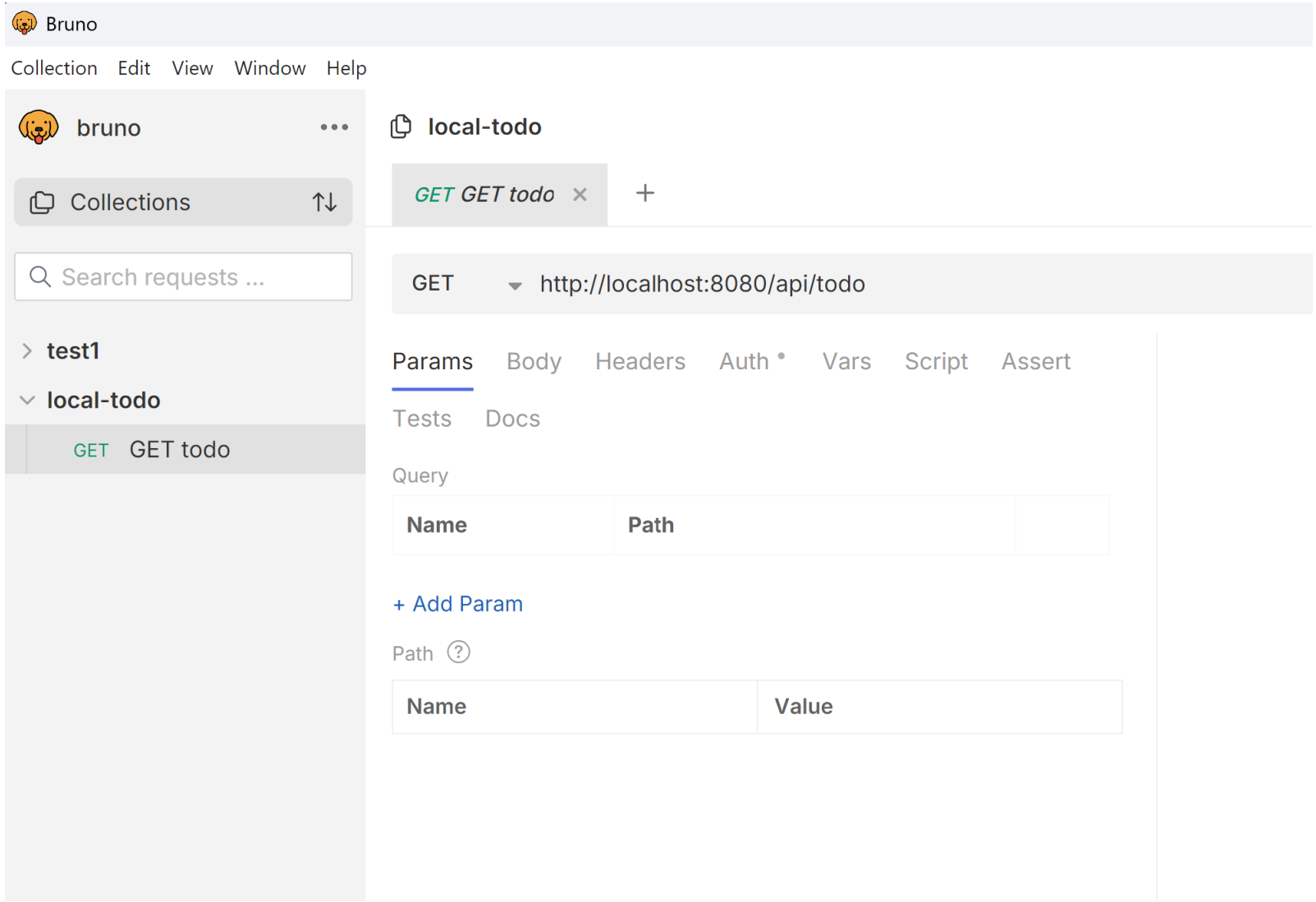
https://usebruno.com



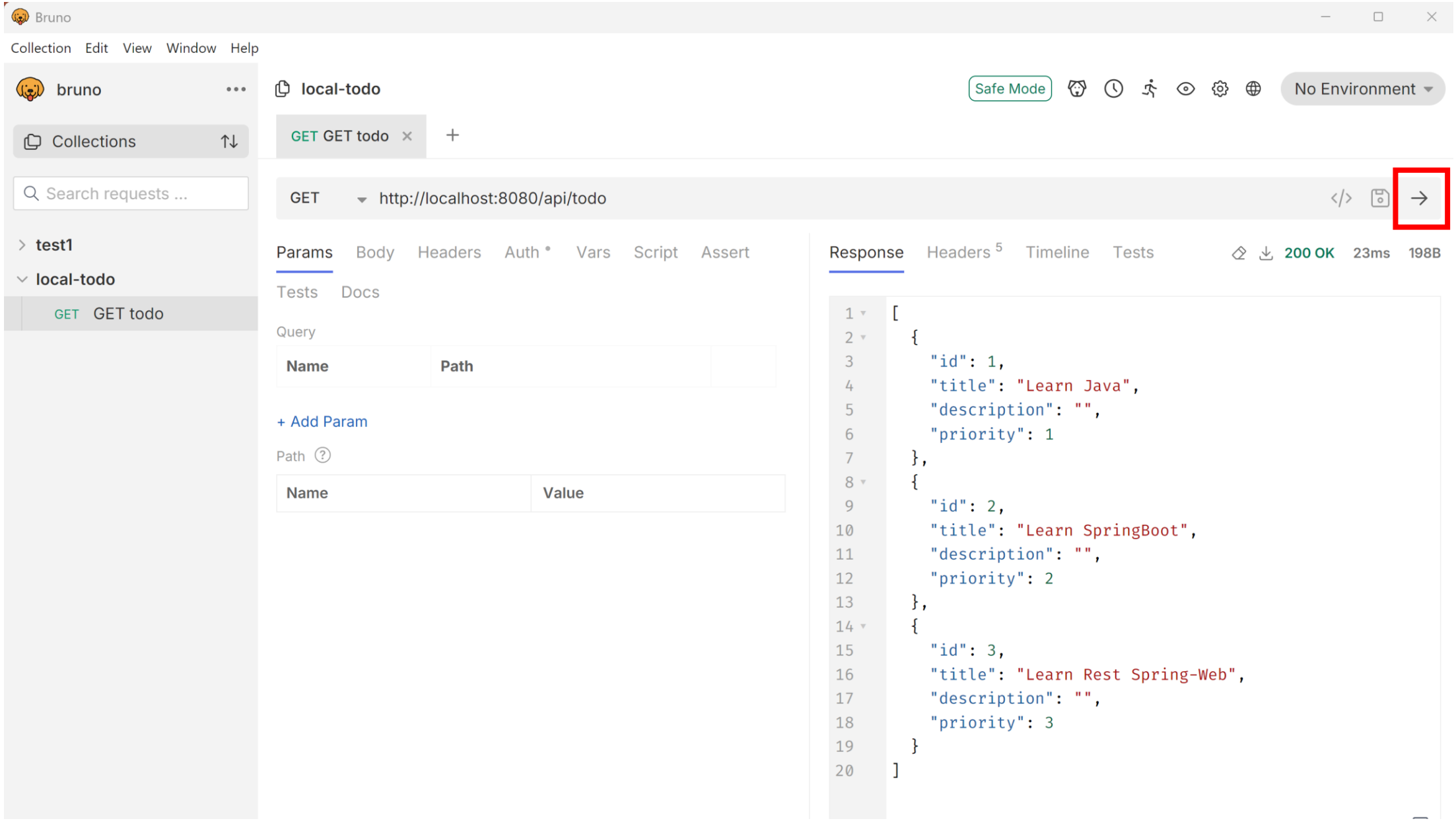
Bruno – Create Request



Bruno – Request



Execute Request



OpenAPI – Swagger

Add OpenAPI and swagger-ui maven dependencies to your pom.xml

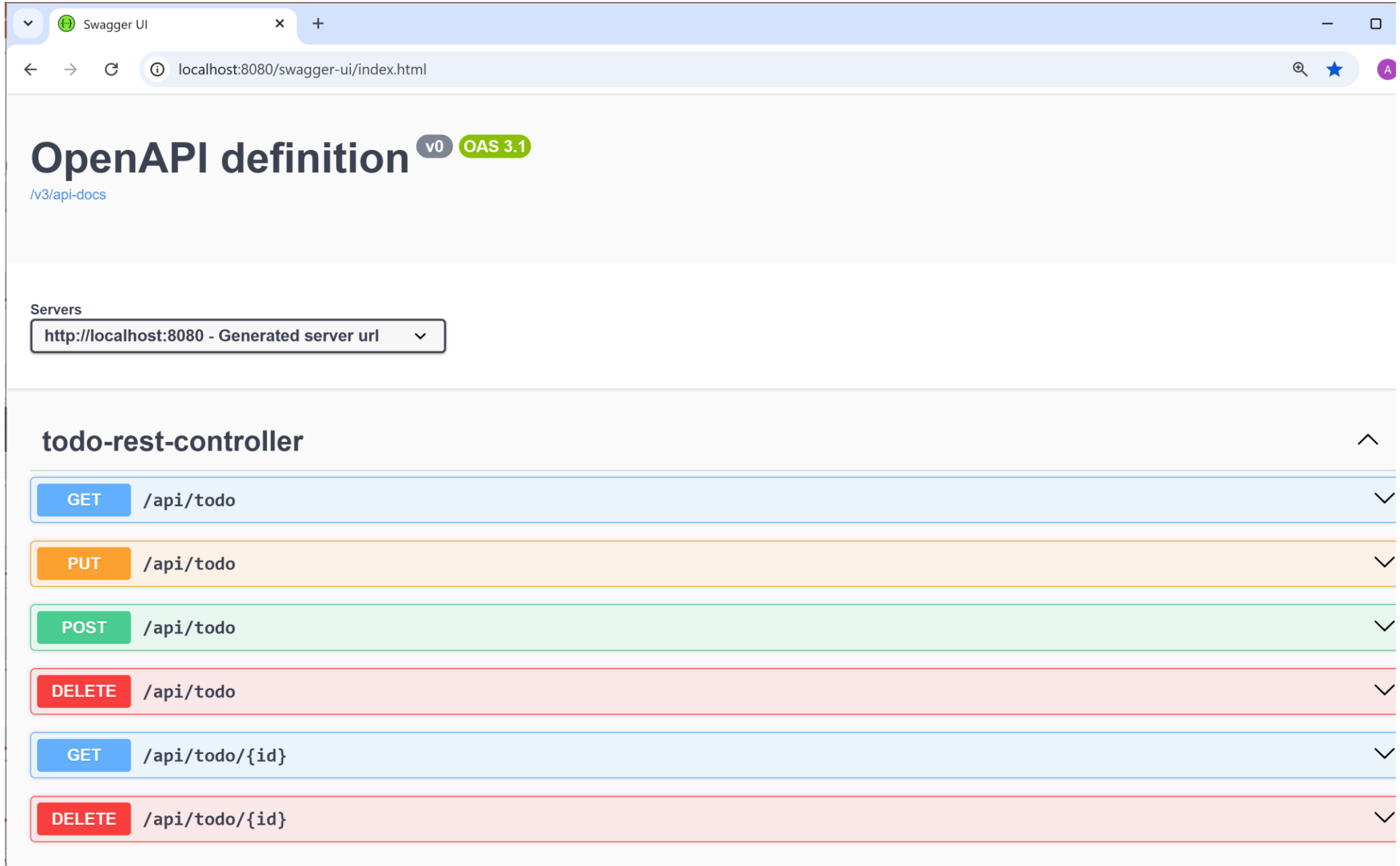
Relaunch

Open « http://localhost:8080/swagger-ui.html »

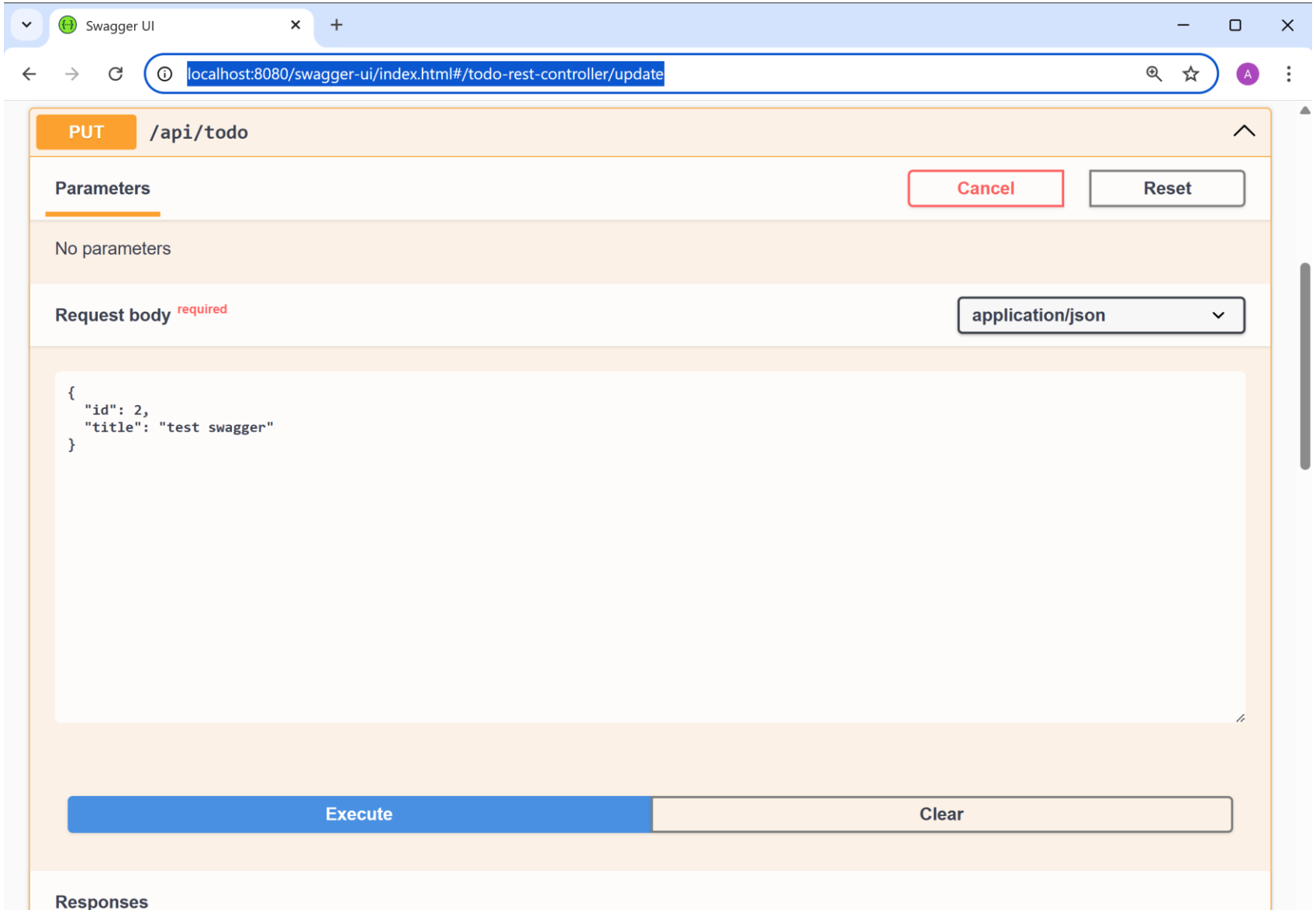
OpenApi…

```
<dependency>
		<groupId>org.springdoc</groupId>
		<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
		<version>2.8.5</version>
</dependency>
```

http://localhost:8080/swagger-ui.html



Execute Request..

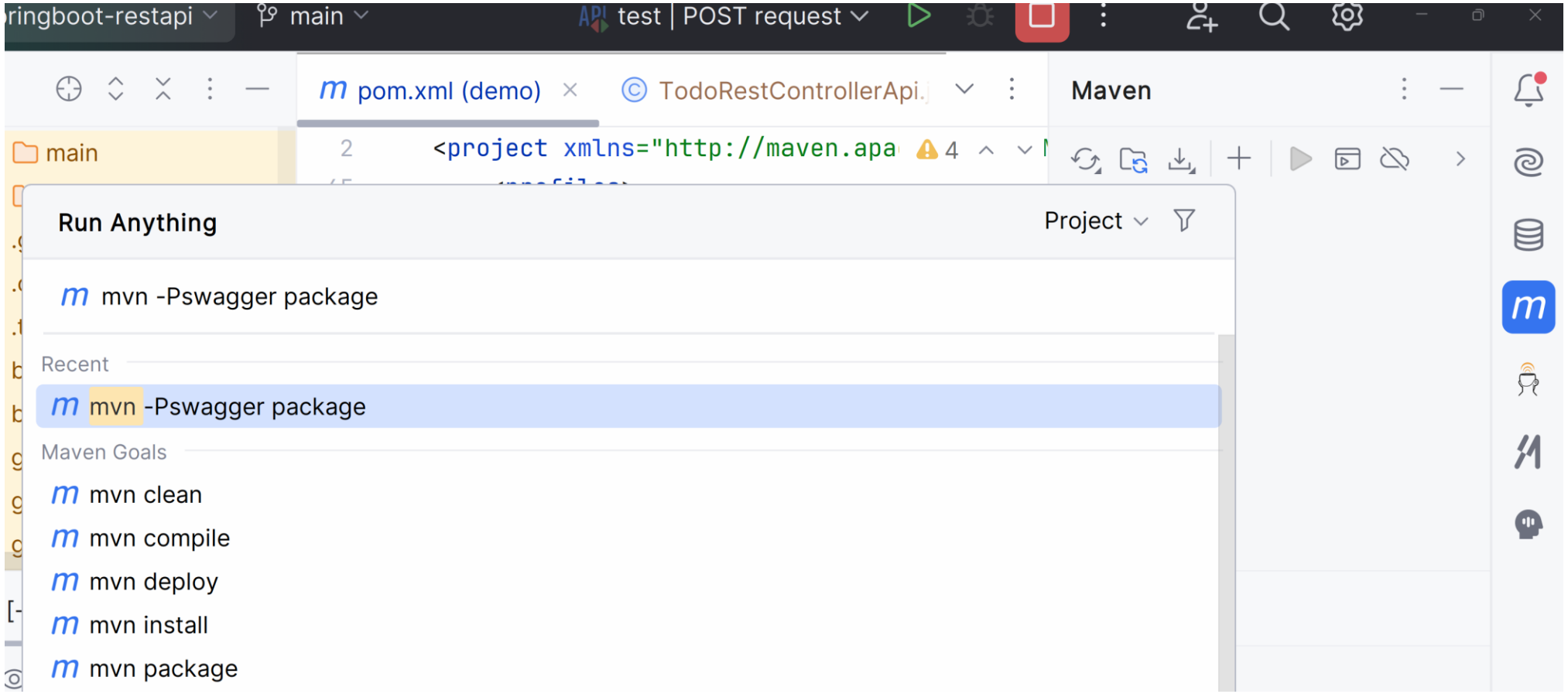


Swagger Code Generator (Maven Plugin) …

```
<build>
        <plugins>
            <plugin>
                <groupId>org.openapitools</groupId>
                <artifactId>openapi-generator-maven-plugin</artifactId>
                <version>7.16.0</version>
                <executions>
                    <execution>
                        <phase>package</phase>
                        <goals>
                            <goal>generate</goal>
                        </goals>
                        <configuration>
                            <inputSpec>http://localhost:8080/v3/api-docs</inputSpec>
                            <generatorName>java</generatorName>
                            <configOptions>
                                <sourceFolder>src/gen/java/main</sourceFolder>
                            </configOptions>
                        </configuration>
                    </execution>
                </executions>
            </plugin>
        </plugins>
</build>
```

Execute Maven code generator

mvn package



Sample Generated Code

